

SAPIENZA
UNIVERSITÀ DI ROMA

Foundations of Data Science

Appunti del corso

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Corso di Laurea Magistrale in Computer Science

Docente
Prof. Indro Spinelli

Appunti di
Alessio Cavedal
GitHub: https://github.com/AlessioCavedal

Anno Accademico 2026/2027

Indice

1 Data science e machine learning **1**
1.1 Che cos'è la data science . . . 1
1.2 Intelligenza artificiale e machine learning . . . 2
1.3 I tre paradigmi dell'apprendimento . . . 3
1.4 Supervised learning . . . 3
1.5 Unsupervised learning . . . 6
1.6 Reinforcement learning . . . 8

2 I dati come vettori **9**
2.1 I dati nel supervised learning . . . 9
2.2 Tipi di dato ed encoding . . . 10
2.3 Il dataset in termini formali . . . 11
2.4 I punti come vettori . . . 12
2.5 Spanning set e basi . . . 12
2.6 La base standard e le basi ortonormali . . . 13
2.7 Basi qualsiasi: encoding e calcolo dei pesi . . . 15
2.8 Sottospazi e proiezioni . . . 17

3 Principal Component Analysis **19**
3.1 Le trasformazioni lineari . . . 19
3.2 Encoding, decoding ed errore di ricostruzione . . . 20
3.3 Imparare la base . . . 20
3.4 Dalla ricostruzione alla varianza . . . 21
3.5 La matrice di covarianza . . . 22
3.6 Le componenti principali . . . 23
3.7 A che cosa serve . . . 24
3.8 Un esempio svolto . . . 25
3.9 Quando funziona: i manifold . . . 26

4 Ottimizzazione **28**
4.1 Minimi e massimi . . . 28
4.2 La condizione del primo ordine . . . 29
4.3 Il gradiente . . . 31
4.4 Calcolare il gradiente . . . 33
4.5 Gradient descent . . . 36
4.6 Il passo e quando fermarsi . . . 38

4.7 I limiti del gradient descent 40
4.8 La PCA con il gradient descent 42

Capitolo 1

Data science e machine learning

1.1 Che cos'è la data science

Definizione (data science). La **data science** è un campo interdisciplinare che estrae dai dati conoscenza e indicazioni concrete su cui agire (*actionable insights*). Comprende la raccolta, la pulizia, la gestione e l'analisi di dati strutturati e non strutturati, con tecniche che vengono dalla statistica, dalla matematica, dall'informatica e dalla conoscenza del dominio applicativo.

Un progetto di data science non è un'analisi isolata, ma un ciclo di cinque fasi che si ripete (Figura 1.1).

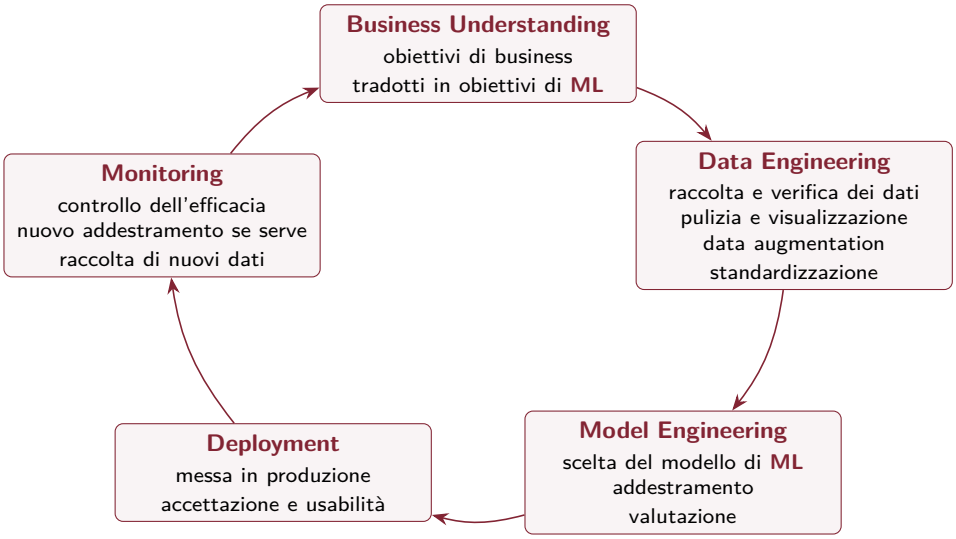


Figura 1.1. Il ciclo di vita di un progetto di data science. ML sta per machine learning; la *data augmentation* arricchisce i dati con esempi nuovi.

Esempio (un servizio in abbonamento). Un servizio di streaming perde troppi abbonati. Nella fase di *business understanding* l'obiettivo «perdere meno clienti» diventa un problema di machine learning: prevedere, per ogni abbonato, se

disdirà nel prossimo mese, così da offrirgli uno sconto in tempo. Nel *data engineering* si raccolgono i dati utili (da quanto è abbonato, quanto guarda, se ha contattato l'assistenza), si eliminano errori e duplicati e si portano tutti i dati sulla stessa scala. Nel *model engineering* si sceglie un modello, lo si addestra sui clienti del passato, di cui si sa chi ha disdetto, e si misura quanto bene prevede. Con il *deployment* le previsioni arrivano ogni giorno al reparto marketing, in una forma che sappia usare. Infine, con il *monitoring*, si controlla che le previsioni restino buone: se le abitudini degli utenti cambiano, si raccolgono dati nuovi e si riaddestra il modello, e il ciclo ricomincia.

Il machine learning entra in due fasi chiave, la formulazione degli obiettivi e la scelta del modello: per questo il corso si concentra su di esso.

1.2 Intelligenza artificiale e machine learning

L'**intelligenza artificiale** (*Artificial Intelligence*, AI) è il campo dell'informatica che si occupa di costruire agenti capaci di svolgere compiti che di solito richiedono l'intelligenza umana, come imparare, ragionare e risolvere problemi. Il **machine learning** (ML, apprendimento automatico) ne studia un aspetto preciso: l'apprendimento dall'esperienza. La definizione classica è di Tom Mitchell.

Definizione (machine learning). Il machine learning studia gli algoritmi che migliorano la propria *prestazione* P in un *compito* T grazie all'*esperienza* E. Un problema di apprendimento ben definito è dato dalla terna $\langle P, T, E \rangle$.

Le tre lettere vengono dai nomi inglesi: *performance*, *task* ed *experience*. La definizione obbliga a dire con precisione che cosa si vuole imparare (T), come si misura il successo (P) e da quali dati si impara (E). La Tabella 1.1 riporta quattro esempi.

Compito T	Prestazione P	Esperienza E
giocare a dama	percentuale di partite vinte contro un avversario qualsiasi	partite di allenamento giocate contro sé stesso
riconoscere parole scritte a mano	percentuale di parole classificate correttamente	database di immagini di parole scritte a mano, etichettate da persone
guidare su un'autostrada a quattro corsie con sensori visivi	distanza media percorsa prima di un errore, giudicato da una persona	sequenza di immagini e di comandi di sterzo registrati osservando un guidatore
classificare le email come spam o legittime	percentuale di email classificate correttamente	database di email, alcune etichettate a mano

Tabella 1.1. Quattro problemi di apprendimento descritti con la terna di Mitchell.

Il machine learning serve perché molti compiti, come riconoscere persone e oggetti in un'immagine o capire il parlato e il linguaggio, sono troppo complessi per scriverne le regole a mano: conviene che sia l'algoritmo a impararle dai dati.

> **Machine learning e statistica.** Machine learning e statistica hanno molto in comune: entrambi cercano regolarità (*pattern*) nei dati, usano analisi, probabilità e algebra lineare, e condividono molti algoritmi. Cambia l'enfasi. La statistica punta sull'inferenza: trarre conclusioni rigorose e interpretabili sul fenomeno che ha prodotto i dati. Il machine learning punta sulla previsione, sulla scalabilità e sull'autonomia del sistema.

1.3 I tre paradigmi dell'apprendimento

Il machine learning si divide in tre paradigmi, a seconda dei dati da cui si impara (Tabella 1.2):

- nel **supervised learning** (apprendimento supervisionato) si hanno esempi etichettati del comportamento corretto, cioè coppie formate da un input e dall'output desiderato. Per esempio, migliaia di email ciascuna con l'indicazione «spam» o «legittima»: il sistema impara a dare l'etichetta giusta alle email nuove;
- nell'**unsupervised learning** (apprendimento non supervisionato) non ci sono etichette, e si cercano nei dati strutture o regolarità «interessanti». Per esempio, dagli acquisti dei clienti di un negozio si scopre che esistono alcuni gruppi con abitudini simili, senza che nessuno abbia detto prima quali siano;
- nel **reinforcement learning** (apprendimento per rinforzo) un sistema che apprende, l'**agente**, interagisce con il mondo e impara ad agire in modo razionale, massimizzando un segnale scalare di ricompensa (*reward*). Per esempio, un programma che gioca a scacchi: nessuno gli dice qual è la mossa giusta, ma vincere o perdere pezzi e partite gli indica se sta giocando bene.

Paradigma	Input	Output
Supervised	esempi di input insieme all'output desiderato	un modello che predice l'output per un input nuovo
Unsupervised	esempi di dati, senza output	una rappresentazione della struttura dei dati
Reinforcement	una sequenza di interazioni con un ambiente	una *policy*, cioè una strategia che svolge il compito desiderato

Tabella 1.2. Input e output dei tre paradigmi dell'apprendimento.

1.4 Supervised learning

Il supervised learning rovescia la programmazione tradizionale (Figura 1.2): lì il programmatore scrive il programma e il computer lo applica ai dati per produrre

l'output; qui si forniscono al computer i dati insieme agli output attesi, ed è il computer a ricavarne il «programma», cioè il modello.

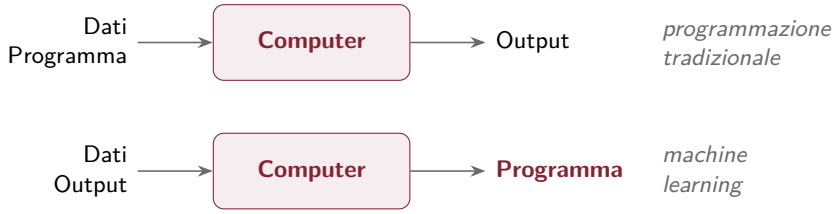


Figura 1.2. Programmazione tradizionale e machine learning a confronto.

1.4.1 Dall'input reale all'output reale

Tutti gli esempi di supervised learning seguono lo stesso schema (Figura 1.3). L'input del mondo reale, che siano le caratteristiche di una casa, un testo o un'immagine, viene trasformato in un vettore di numeri, l'input del modello. Il modello produce un altro vettore, l'output del modello, che si interpreta di nuovo nel mondo reale: un prezzo, una classe, una frase. Un modello lavora soltanto su vettori di numeri: come si rappresentano i dati in questa forma è l'argomento del Capitolo 2.

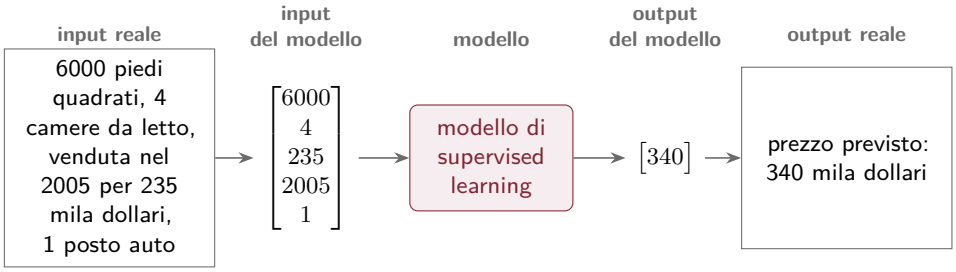


Figura 1.3. Lo schema comune agli esempi di supervised learning, nella stima del prezzo di una casa.

1.4.2 Tipi di problema

In base all'output si distinguono:

- la **regressione**, in cui l'output è un numero reale: è **univariata** se l'output è uno solo, **multivariata** se sono più d'uno;
- la **classificazione**, in cui l'output è una classe scelta in un insieme discreto: è **binaria** se le classi sono due, **multiclasse** se sono più di due;
- la **classificazione binaria multivariata**, con molti output a due classi: per esempio decidere, pixel per pixel, se un'immagine mostra un oggetto o lo sfondo.

La Tabella 1.3 raccoglie alcuni esempi, con l'architettura di rete neurale usata: per ora basta sapere che compiti diversi richiedono modelli diversi. Gli ultimi tre non rientrano nei tipi elencati sopra: l'output non è un numero né una classe, ma un oggetto strutturato (una frase, un'immagine), e le risposte valide possono essere molte.

Compito	Input → output	Tipo di problema	Architettura
regressione	caratteristiche di una casa → prezzo	regressione univariata	rete *fully connected*
graph regression	molecola → temperatura di congelamento e di ebollizione	regressione multivariata	*graph neural network*
classificazione di testi	recensione di un ristorante → positiva o negativa	classificazione binaria	*transformer*
genere musicale	brano audio → genere (classica, jazz, pop...)	classificazione multiclasse	rete ricorrente (RNN)
image classification	immagine → oggetto raffigurato (bicicletta, ape...)	classificazione multiclasse	rete convoluzionale
image segmentation	immagine → per ogni pixel, oggetto o sfondo	classificazione binaria multivariata	*encoder-decoder* convoluzionale
depth estimation	immagine → profondità di ogni pixel	regressione multivariata	*encoder-decoder* convoluzionale
pose estimation	immagine di una persona → posizione delle articolazioni	regressione multivariata	*encoder-decoder* convoluzionale
traduzione	testo in inglese → testo in francese	output strutturato	–
image captioning	immagine → frase che la descrive	output strutturato	–
generazione di immagini	descrizione testuale → immagine	output strutturato	–

Tabella 1.3. Esempi di supervised learning, con il tipo di problema e l'architettura di rete.

Nei problemi di classificazione il modello di solito non restituisce la classe, ma un valore per ciascuna: per una recensione, per esempio, 0,02 per «positiva» e 0,98 per «negativa». Si sceglie la classe con il valore più alto.

1.4.3 Che cos'è un modello

Un modello di supervised learning è, in sostanza, un'equazione che lega l'input all'output. Nell'esempio della Figura 1.4 l'input è l'età di un bambino e l'output la sua altezza: per un bambino di 10 anni il modello prevede 139 cm. Addestrare il modello significa cercare, in una *famiglia* di equazioni possibili, quella che si adatta bene ai dati di addestramento (*training data*), i punti del grafico.

Per capire che cosa sia una famiglia di equazioni, si pensi alle rette altezza $=$ $a + b \cdot$ età. Ogni coppia di numeri a e b, i **parametri**, dà una retta diversa: con $a = 80$ e $b = 6$ un bambino di 10 anni risulterebbe alto 140 cm, con $a = 50$ e $b = 2$ appena 70 cm. Addestrare il modello vuol dire scegliere i valori di a e b che sbagliano di meno sui bambini di cui si conosce già l'altezza; poi il modello si usa su quelli nuovi. Le rette però sono una famiglia povera: la crescita non è lineare (rallenta

dopo l'adolescenza), e nessuna retta segue bene i punti della figura. Per questo la curva disegnata viene da una famiglia più flessibile.

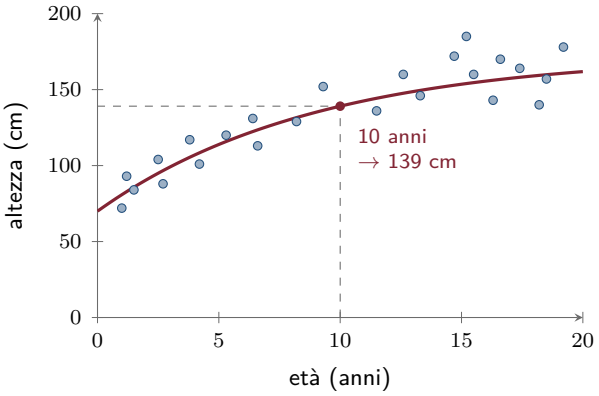


Figura 1.4. Un modello come equazione: la curva lega l'età all'altezza, e i punti sono i dati di addestramento.

Le reti neurali profonde (*deep neural networks*) sono semplicemente una famiglia di equazioni molto più flessibile: l'idea è la stessa delle rette, ma invece di due parametri ce ne sono milioni, abbastanza per compiti come riconoscere un oggetto in un'immagine. Adattarle ai dati è ciò che si chiama **deep learning**.

1.4.4 Che cosa hanno in comune gli esempi

Gli esempi della Tabella 1.3 hanno tre tratti comuni:

- la relazione tra input e output è molto complessa;
- a volte le risposte valide sono molte: una frase si traduce in più modi, e una descrizione corrisponde a infinite immagini;
- gli output, e talvolta gli input, obbediscono a delle regole: il linguaggio rispetta una grammatica, e anche le immagini naturali hanno le loro «regole», perché non ogni combinazione di pixel sembra una fotografia.

Da qui un'idea: imparare la «grammatica» dei dati da esempi senza etichetta, che si possono raccogliere in quantità enormi proprio perché nessuno deve etichettarli. Sapendo bene quali output sono plausibili, il compito supervisionato diventa più facile. È uno dei motivi per cui l'apprendimento non supervisionato è così importante.

1.5 Unsupervised learning

I compiti tipici dell'unsupervised learning sono:

- il **clustering**, cioè raggruppare i dati simili. Un esempio è DeepCluster, che raggruppa le immagini con caratteristiche visive simili senza sapere che cosa raffigurano;
- trovare gli **outlier**, i dati anomali;

- generare nuovi esempi, simili a quelli del dataset;
- ricostruire i dati mancanti.

Una classe importante è quella dei **modelli generativi**, capaci di creare nuovi esempi: da migliaia di foto di gatti, per esempio, imparano a produrre foto di gatti che non esistono. Ne sono un esempio le *generative adversarial networks* (GAN). Al loro interno, i **modelli generativi probabilistici** (*probabilistic generative models*, PGM) imparano esplicitamente una distribuzione di probabilità sui dati: tra questi i *variational autoencoder* (VAE), i *normalizing flow* e i *diffusion model*. I tre insiemi sono uno dentro l'altro (Figura 1.5).

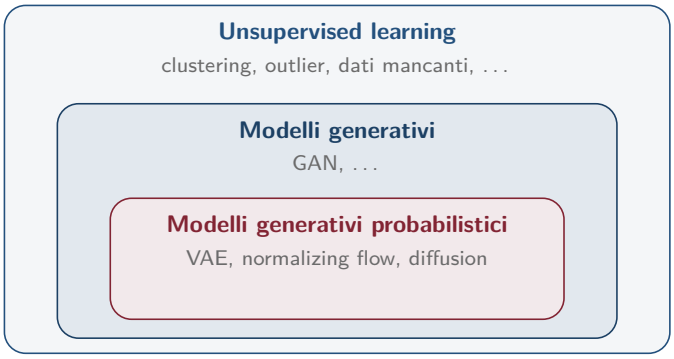


Figura 1.5. I modelli generativi sono una parte dell'unsupervised learning, e i modelli generativi probabilistici una parte dei modelli generativi.

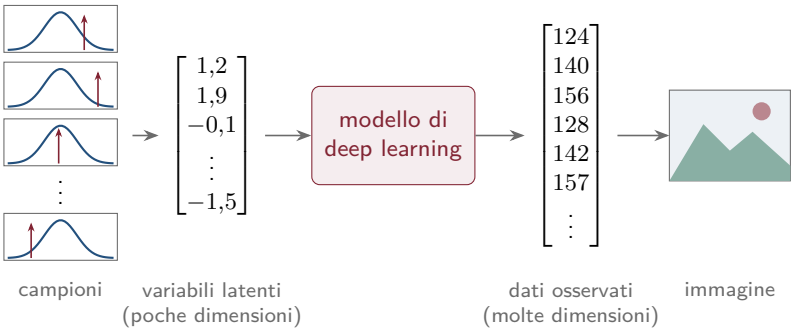


Figura 1.6. Dalle variabili latenti a un dato ad alta dimensione.

Molti modelli generativi si basano sulle **variabili latenti** (Figura 1.6): si estraggono a caso pochi numeri da distribuzioni semplici, per esempio gaussiane, e un modello di deep learning li trasforma in un dato osservato ad alta dimensione, come un'immagine. L'idea è che dati complessi, fatti di moltissimi numeri, dipendano da pochi fattori nascosti. Una foto di un volto, per esempio, è fatta di centinaia di migliaia di pixel, ma ciò che la determina è poco: la persona, la posa, la luce, l'espressione. Le variabili latenti giocano il ruolo di questi fattori, anche se il modello li impara da solo e non hanno per forza un significato così chiaro.

1.6 Reinforcement learning

Nel reinforcement learning ci sono un insieme di **stati**, un insieme di **azioni** e un insieme di **ricompense**. L'obiettivo è scegliere le azioni in modo da cambiare lo stato e ricevere ricompense. A differenza degli altri due paradigmi non si riceve nessun dataset: l'agente deve esplorare l'ambiente e raccogliere i dati da sé, man mano che agisce (Figura 1.7).

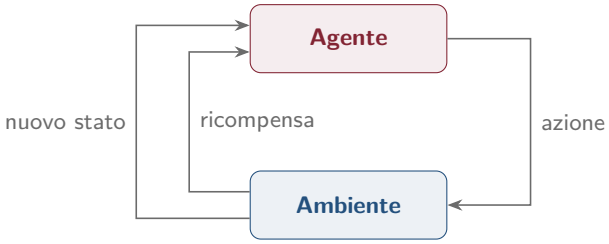


Figura 1.7. Il ciclo del reinforcement learning: l'agente sceglie un'azione, l'ambiente risponde con un nuovo stato e una ricompensa.

Esempio (gli scacchi). Per il programma che gioca a scacchi gli stati sono le configurazioni valide della scacchiera e le azioni, in un dato momento, sono le mosse consentite. Le ricompense sono positive quando si mangia un pezzo dell'avversario e negative quando se ne perde uno.

Il problema è difficile per tre motivi:

- è **stocastico**: la stessa mossa ripetuta due volte può avere esiti diversi, perché l'avversario può rispondere in un altro modo, e anche le ricompense sono incerte (l'avversario può mangiare un pezzo oppure no);
- c'è il **temporal credit assignment problem** (attribuzione temporale del merito): quando arriva una ricompensa, non è chiaro se sia merito dell'ultima mossa o di buone scelte tattiche fatte molto prima;
- c'è l'**exploration-exploitation trade-off** (compromesso tra esplorazione e sfruttamento): se si è trovata una buona apertura, conviene usare sempre quella o provarne altre, nella speranza di trovarne una migliore?

Capitolo 2

I dati come vettori

Un modello lavora soltanto su vettori di numeri: prima di costruirne uno bisogna quindi capire come sono fatti i dati e come li si trasforma in vettori.

2.1 I dati nel supervised learning

Un esempio classico è il dataset **IRIS**: 150 fiori di iris, 50 per ciascuna di tre specie (*setosa*, *versicolor* e *virginica*), di cui sono misurate in centimetri lunghezza e larghezza del sepalo e del petalo. Il compito è riconoscere la specie a partire dalle quattro misure.

In generale i dati del supervised learning sono misure di diversi aspetti di P oggetti, raccolte in una tabella con una riga per oggetto (Tabella 2.1). Le colonne sono di due tipi:

- il **target** è la variabile di output, l'obiettivo della previsione: qui la specie;
- le **feature** sono le proprietà che descrivono l'oggetto: qui le quattro misure.

Si assume che tra feature e target ci sia una relazione, e si vuole scoprire la regola predittiva che la descrive.

Feature				Target
Sepal.Length	*Sepal.Width*	*Petal.Length*	*Petal.Width*	*Species*
4,3	3,0	1,1	0,1	setosa
5,0	3,3	1,4	0,2	setosa
7,7	3,8	6,7	2,2	virginica
5,5	2,5	4,0	1,3	versicolor

Tabella 2.1. Quattro righe del dataset IRIS: le misure in centimetri sono le feature, la specie è il target.

Le voci della colonna del target si chiamano **label** (etichette), e si distinguono due tipi di dati:

- i **dati etichettati** (*labeled data*), in cui il target è osservato;
- i **dati non etichettati** (*unlabeled data*), in cui il target è sconosciuto.

Il supervised learning impara dai primi; l'unsupervised learning (Sezione 1.5) lavora sui secondi.

2.2 Tipi di dato ed encoding

Feature e target possono essere di tipi diversi:

- **numerici**, con valori in $\mathbb{R}$, l'insieme dei numeri reali;
- **interi**, con valori in $\mathbb{Z}$, l'insieme dei numeri interi (la lettera viene dal tedesco *Zahlen*, «numeri»);
- **categorici**, con valori in un insieme finito di categorie, come le tre specie di IRIS;
- **binari**, con valori in $\{0, 1\}$.

Il tipo del target decide il tipo di problema (Sezione 1.4): un target numerico dà un problema di regressione, un target categorico un problema di classificazione. IRIS, con la specie come target, è un problema di classificazione con tre classi.

Quasi tutti gli algoritmi di apprendimento lavorano solo con feature numeriche, salvo qualche eccezione. Le feature degli altri tipi vanno quindi convertite in numeri: bisogna scegliere, o costruire, un **encoding** (codifica) adatto.

Definizione (one-hot encoding). Con l'**encoding one-hot**, una variabile categorica con k categorie mutuamente esclusive passa da un singolo valore a un vettore binario di k elementi, con un solo elemento uguale a 1, nella posizione della categoria, e 0 altrove. La categoria i-esima diventa

$$\mathbf{e}_i = (0, \ldots, 0, \underbrace{1}_{\text{posizione } i}, 0, \ldots, 0)^\top, \qquad i \in \{1, \ldots, k\}.$$

Il simbolo $^\top$, la **trasposta**, scambia righe e colonne: il vettore è scritto in riga per risparmiare spazio, ma va letto in colonna, che è il modo in cui l'algebra lineare scrive di solito i vettori.

Esempio (le specie di IRIS). Con le specie nell'ordine setosa, versicolor, virginica, l'encoding one-hot è (la freccia $\mapsto$ si legge «diventa»)

$$\text{setosa} \mapsto (1, 0, 0)^\top, \qquad \text{versicolor} \mapsto (0, 1, 0)^\top, \qquad \text{virginica} \mapsto (0, 0, 1)^\top.$$

Se invece le categorie hanno un ordine naturale, come «basso», «medio» e «alto», si usa un encoding che lo rispecchi, per esempio una sequenza di interi: 1, 2, 3. Per categorie senza ordine, come le specie, gli interi sarebbero fuorvianti, perché introdurrebbero un ordine e delle distanze che non esistono (versicolor non sta «tra» setosa e virginica); i vettori one-hot invece sono tutti alla stessa distanza l'uno dall'altro.

2.3 Il dataset in termini formali

La tabella dei dati ha anche una scrittura matematica, che conviene vedere subito su IRIS. Ogni riga è un'**osservazione**, cioè una coppia $(\mathbf{x}_p, y_p)$: $\mathbf{x}_p$ è il vettore delle feature del fiore numero p, y_p la sua specie. Per la prima riga della Tabella 2.1

$$\mathbf{x}_1 = (4{,}3;\ 3{,}0;\ 1{,}1;\ 0{,}1), \qquad y_1 = \text{setosa}.$$

Il singolo valore ha due indici, prima la feature e poi il fiore: $x_{n,p}$ è la feature n del fiore p, quindi $x_{3,1} = 1{,}1$ è la terza feature (la lunghezza del petalo) del primo fiore. Una colonna della tabella raccoglie invece una sola feature per tutti i fiori: la prima contiene le 150 lunghezze del sepalo, una per fiore.

Il **dataset** è l'insieme di tutte le osservazioni,

$$D = ((\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_P, y_P)),$$

con $P = 150$ per IRIS. I vettori $\mathbf{x}_p$ stanno nello **spazio degli input** X. Quando le N feature sono numeriche (per IRIS $N = 4$), di solito $X \subseteq \mathbb{R}^N$: $\mathbb{R}^N$ è l'insieme di tutti i vettori di N numeri reali, e il simbolo $\subseteq$ dice che X ne è una parte. I target stanno nello **spazio degli output** Y, che per IRIS è l'insieme delle tre specie.

2.3.1 Il processo che genera i dati

I 150 fiori di IRIS non sono tutti i fiori di iris che esistono: sono un campione. Si immagina che dietro i dati ci sia un processo che li genera, descritto da una distribuzione di probabilità $\mathbb{P}_{XY}$ sulle coppie (input, target): dice quanto è probabile incontrare un fiore con certe misure e di una certa specie. Le quantità estratte a caso da questa distribuzione, le **variabili aleatorie**, si indicano con le minuscole $\mathbf{x}$ e y.

Che serva una distribuzione, e non una semplice funzione da $\mathbf{x}$ a y, si capisce dai dati stessi: due fiori con le stesse misure possono essere di specie diverse, perché le specie si sovrappongono un po', e ogni misura ha un errore. Dato $\mathbf{x}$, quindi, la specie non è determinata con certezza: alcune specie sono solo più probabili di altre.

La **distribuzione vera** $\mathbb{P}_{XY}$ di solito è sconosciuta, e si vedono solo i campioni che ne sono usciti. Il supervised learning cerca di impararne la parte che serve per prevedere y a partire da $\mathbf{x}$.

> **Definizione (ipotesi i.i.d.).** Si assume che le osservazioni siano estratte da $\mathbb{P}_{XY}$ in modo **i.i.d.** (*independent and identically distributed*, indipendenti e identicamente distribuite): tutte provengono dalla stessa distribuzione, e ciascuna non dipende dalle altre.

Per IRIS l'ipotesi è ragionevole se i fiori sono stati raccolti a caso: misurare un fiore non dice nulla sul successivo, e tutti vengono dalla stessa popolazione. Non vale invece per il prezzo di un'azione giorno per giorno, perché il prezzo di oggi dipende molto da quello di ieri: è una serie temporale. È quindi un'ipotesi forte e non sempre realistica; esistono scenari più complessi che ne fanno a meno, ma qui ci si limita al caso i.i.d.

2.4 I punti come vettori

D'ora in poi si guarda solo agli input, ignorando il target: ogni osservazione si riduce al suo vettore $\mathbf{x}_p$, che è un punto dello spazio degli input. Si assume inoltre che il dataset sia **centrato** (*mean-centered*): a ogni punto si sottrae la media del dataset, dimensione per dimensione,

$$\bar{\mathbf{x}} = \frac{1}{P}\sum_{p=1}^{P}\mathbf{x}_p, \qquad \mathbf{x}_p \leftarrow \mathbf{x}_p - \bar{\mathbf{x}},$$

così che i dati si dispongano attorno all'origine. La media $\bar{\mathbf{x}}$ è il punto medio: si sommano tutti i punti, componente per componente, e si divide per P; la freccia vuol dire che ogni punto viene sostituito dalla sua differenza con la media. In IRIS, per esempio, la lunghezza media del sepalo è circa 5,8 cm: dopo la centratura un fiore con il sepalo di 4,3 cm ha valore $-1{,}5$, che si legge come «1,5 cm più corto della media». È un'operazione semplice e del tutto reversibile: basta sommare di nuovo la media.

Da qui in poi i punti si trattano con gli strumenti dell'algebra lineare. Negli appunti questi strumenti sono spiegati in breve dove servono; il *Ripasso di matematica*, un PDF a parte, li riprende da zero con esempi ed esercizi. Uno **spazio vettoriale** è un insieme di oggetti, i **vettori**, che si possono sommare tra loro e moltiplicare per numeri, gli **scalari**; $\mathbb{R}^N$ ne è l'esempio tipico. Un punto di $\mathbb{R}^N$ si può disegnare come un punto o come una freccia che parte dall'origine (Figura 2.1): sono due modi di vedere lo stesso oggetto, il primo più comodo per guardare un dataset, il secondo per ragionare su somme e multipli.

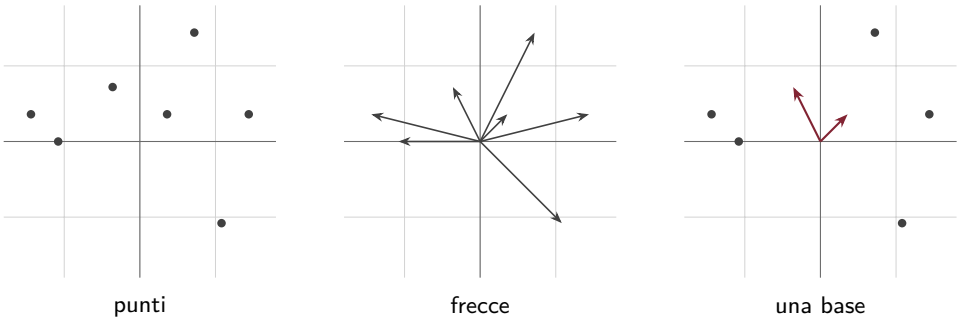


Figura 2.1. Lo stesso dataset in $\mathbb{R}^2$ visto come punti (a sinistra) e come frecce che partono dall'origine (al centro). A destra due soli vettori, in rosso, da cui si ottengono tutti gli altri punti.

2.5 Spanning set e basi

Una **combinazione lineare** di alcuni vettori è una somma di loro multipli, come $2\mathbf{a} + 3\mathbf{b}$: ogni vettore viene moltiplicato per un numero, il suo peso, e i risultati si sommano componente per componente. Con $\mathbf{a} = (1, 0)^\top$ e $\mathbf{b} = (1, 1)^\top$, per esempio, $2\mathbf{a} + 3\mathbf{b} = (2, 0)^\top + (3, 3)^\top = (5, 3)^\top$.

Definizione (spanning set). Uno **spanning set** (insieme di generatori) è un insieme di vettori le cui combinazioni lineari producono *ogni* vettore dello spazio. Si dice che i vettori **generano** (*span*) lo spazio.

Per esempio, in $\mathbb{R}^2$ i vettori $\mathbf{e}_1 = (1,0)^\top$ e $\mathbf{e}_2 = (0,1)^\top$ generano il piano, perché ogni punto (x, y) si scrive come $x\mathbf{e}_1 + y\mathbf{e}_2$: $(3,1)$ è $3\mathbf{e}_1 + 1\mathbf{e}_2$.

Definizione (base). Una **base** è uno spanning set minimale: il più piccolo insieme di vettori che genera ancora lo spazio. Nessun vettore deve essere ridondante, cioè i vettori devono essere **linearmente indipendenti**: nessuno si può ottenere come combinazione lineare degli altri.

In $\mathbb{R}^2$ la coppia $\{\mathbf{e}_1, \mathbf{e}_2\}$ è una base, e lo sono anche i due vettori rossi della Figura 2.1. Se a $\mathbf{e}_1$ e $\mathbf{e}_2$ si aggiunge $\mathbf{e}_1 + \mathbf{e}_2 = (1,1)^\top$, i tre vettori generano ancora il piano, ma il terzo è ridondante, perché è la somma degli altri due. La conseguenza pratica è che i modi di scrivere un punto diventano infiniti: $(2,1)$ è $2\mathbf{e}_1 + \mathbf{e}_2$, ma anche $\mathbf{e}_1 + (\mathbf{e}_1 + \mathbf{e}_2)$, e così via. Con una base invece la scrittura è unica, ed è questo che la rende utile: ogni punto ha un solo insieme di «coordinate».

Perché una base $\mathbf{c}_1, \ldots, \mathbf{c}_N$ possa rappresentare perfettamente tutti i P punti del dataset, deve vivere nello stesso spazio $\mathbb{R}^N$ dei dati, e per ogni punto deve esistere un insieme di **pesi** con cui la combinazione lineare dei vettori della base restituisce il punto:

$$\sum_{n=1}^{N} \mathbf{c}_n w_{n,p} = \mathbf{x}_p, \qquad p = 1, \ldots, P. \tag{2.1}$$

La somma $\sum_{n=1}^{N}$ dice di moltiplicare ogni vettore della base per il suo peso e di sommare i risultati. Il peso $w_{n,p}$ dice quanto del vettore $\mathbf{c}_n$ serve per ricostruire il punto $\mathbf{x}_p$: il primo indice sceglie il vettore della base, il secondo il punto. Con la base $\mathbf{e}_1, \mathbf{e}_2$ del piano, per esempio, il punto $\mathbf{x}_p = (3,1)^\top$ ha pesi $w_{1,p} = 3$ e $w_{2,p} = 1$.

2.6 La base standard e le basi ortonormali

La base più semplice è la **base standard**: il vettore $\mathbf{c}_n$ ha tutti zeri tranne un 1 nella posizione n, cioè $\mathbf{c}_n = (0, \ldots, 0, 1, 0, \ldots, 0)^\top$. Per descriverne le proprietà servono due strumenti:

- il **prodotto scalare** (*inner product*) $\mathbf{a}^\top\mathbf{b}$ moltiplica le componenti dei due vettori nella stessa posizione e somma i risultati: per $\mathbf{a} = (1,2)^\top$ e $\mathbf{b} = (3,4)^\top$ vale $1 \cdot 3 + 2 \cdot 4 = 11$. Il risultato è un numero, e vale zero quando i vettori sono perpendicolari, o **ortogonali**: $(1,1)^\top$ e $(-1,1)^\top$ danno $-1 + 1 = 0$;
- la **norma** $\|\mathbf{a}\|_2$ è la lunghezza del vettore, data dal teorema di Pitagora: $\|(3,4)^\top\|_2 = \sqrt{3^2 + 4^2} = 5$. Il pedice 2 indica questa lunghezza «in linea d'aria», e il quadrato toglie la radice: $\|\mathbf{a}\|_2^2 = \mathbf{a}^\top\mathbf{a}$ è la somma dei quadrati delle componenti.

Le proprietà chiave della base standard sono allora due: vettori diversi sono ortogonali, e ogni vettore ha lunghezza 1,

$$\mathbf{c}_n^\top \mathbf{c}_m = 0 \quad \text{per } n \neq m, \qquad \|\mathbf{c}_n\|_2^2 = 1 \quad \text{per } n = 1, \dots, N.$$

Rappresentare un punto sulla base standard è banale: il vettore dei pesi $\mathbf{w}_p = (w_{1,p}, \dots, w_{N,p})^\top$ coincide con il punto stesso, $\mathbf{w}_p = \mathbf{x}_p$, cioè ogni peso è il valore della coordinata corrispondente (Figura 2.2, a sinistra). I vettori one-hot della Sezione 2.2 sono proprio i vettori della base standard di $\mathbb{R}^k$.

Conviene mettere i vettori della base uno accanto all'altro, come colonne di una tabella di numeri: la **matrice** $\mathbf{C}$, con N righe e N colonne. Le due proprietà si scrivono allora in forma compatta:

$$\mathbf{C}^\top \mathbf{C} = \mathbf{I}_{N \times N}.$$

Qui $\mathbf{C}^\top$ è la trasposta di $\mathbf{C}$, che ha in riga i vettori della base, e il prodotto tra due matrici combina righe e colonne: l'elemento di posto (n, m) di $\mathbf{C}^\top \mathbf{C}$ è il prodotto scalare tra la riga n di $\mathbf{C}^\top$, cioè $\mathbf{c}_n$, e la colonna m di $\mathbf{C}$, cioè $\mathbf{c}_m$ (è lo stesso schema di $\mathbf{a}^\top \mathbf{b}$: una riga per una colonna). Il risultato vale 1 sulla diagonale e 0 altrove: è la matrice **identità** $\mathbf{I}$, che moltiplicata per un vettore lo lascia com'è. Un insieme di vettori con questa proprietà si dice **ortonormale**: ortogonali tra loro e di lunghezza 1.

La base standard non è l'unica base ortonormale: ruotando gli assi se ne ottengono altre, e con tutte i pesi si trovano senza fatica. Il motivo è geometrico. Il prodotto scalare $\mathbf{c}_n^\top \mathbf{x}_p$ tra un vettore di lunghezza 1 e un punto è la lunghezza dell'«ombra» del punto sulla retta di $\mathbf{c}_n$, cioè quanto il punto avanza nella direzione di $\mathbf{c}_n$. Se gli assi sono perpendicolari tra loro, queste ombre non si disturbano a vicenda, e sono esattamente i pesi:

$$w_{n,p} = \mathbf{c}_n^\top \mathbf{x}_p, \qquad \text{cioè} \qquad \mathbf{w}_p = \mathbf{C}^\top \mathbf{x}_p.$$

La seconda scrittura fa tutti i prodotti scalari in un colpo solo: una matrice per un vettore dà il vettore dei prodotti scalari di ogni riga della matrice con il vettore, e le righe di $\mathbf{C}^\top$ sono proprio i vettori della base.

Non c'è nessun sistema di equazioni da risolvere. Il vettore dei pesi si chiama **encoding** del punto, e il passo inverso è il prodotto $\mathbf{C}\mathbf{w}_p$, che si legge per colonne: è la somma delle colonne di $\mathbf{C}$, ciascuna moltiplicata per il suo peso in $\mathbf{w}_p$, cioè il membro sinistro della (2.1), che in forma di matrice si scrive $\mathbf{C}\mathbf{w}_p = \mathbf{x}_p$. Rimettendo l'encoding nella rappresentazione si ritrova quindi il punto di partenza, $\mathbf{C}\mathbf{C}^\top \mathbf{x}_p = \mathbf{x}_p$: $\mathbf{C}^\top$ codifica, $\mathbf{C}$ decodifica.

Qui encoding ha un senso diverso da quello della Sezione 2.2, dove trasformava una categoria in numeri, come setosa in $(1, 0, 0)^\top$: qui il punto è già fatto di numeri, e l'encoding lo riscrive rispetto alla base.

Esempio (assi ruotati di 45°). I vettori $\mathbf{c}_1 = (1, 1)^\top/\sqrt{2}$ e $\mathbf{c}_2 = (-1, 1)^\top/\sqrt{2}$ hanno lunghezza 1 ($\|\mathbf{c}_1\|_2^2 = (1 + 1)/2 = 1$, e lo stesso per $\mathbf{c}_2$) e sono perpendicolari ($\mathbf{c}_1^\top \mathbf{c}_2 = (-1 + 1)/2 = 0$): sono gli assi standard ruotati di 45°. Messi in

colonna,

$$\mathbf{C} = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}, \qquad \mathbf{C}^\top\mathbf{C} = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} = \frac{1}{2}\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} = \mathbf{I}\,:$$

l'elemento in alto a destra, per esempio, è la prima riga di $\mathbf{C}^\top$ per la seconda colonna di $\mathbf{C}$, $\frac{1}{2}(1\cdot(-1)+1\cdot 1) = 0$. Per il punto $\mathbf{x} = (3,1)^\top$ i pesi sono due prodotti scalari, cioè le due componenti di $\mathbf{C}^\top\mathbf{x}$:

$$w_1 = \mathbf{c}_1^\top\mathbf{x} = \frac{3+1}{\sqrt{2}} = 2\sqrt{2}, \qquad w_2 = \mathbf{c}_2^\top\mathbf{x} = \frac{-3+1}{\sqrt{2}} = -\sqrt{2}.$$

La verifica: $2\sqrt{2}\,\mathbf{c}_1 - \sqrt{2}\,\mathbf{c}_2 = (2,2)^\top + (1,-1)^\top = (3,1)^\top$. Il punto avanza molto lungo $\mathbf{c}_1$ e un po' all'indietro lungo $\mathbf{c}_2$.

2.7 Basi qualsiasi: encoding e calcolo dei pesi

Se la base non è quella standard, il vettore dei pesi $\mathbf{w}_p$ è diverso dal punto $\mathbf{x}_p$: è la sua nuova rappresentazione, il suo encoding, nel **transformed feature space** (spazio delle feature trasformato), lo spazio i cui assi coordinati sono i vettori della base. Descrive lo stesso punto, ma «misurato» lungo direzioni diverse.

Si prenda per esempio, in $N = 2$ dimensioni, la base

$$\mathbf{c}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad \mathbf{c}_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

I due vettori sono linearmente indipendenti, perché puntano in direzioni diverse, quindi qualunque punto del piano si rappresenta come loro combinazione lineare:

$$w_{1,p}\mathbf{c}_1 + w_{2,p}\mathbf{c}_2 = \mathbf{x}_p,$$

con pesi $w_{1,p}$ e $w_{2,p}$ che sono unici per ogni punto $\mathbf{x}_p$. La Figura 2.2 confronta le due rappresentazioni dello stesso punto.

Esempio (calcolo dei pesi). Per il punto $\mathbf{x} = (5,5)^\top$ (omettendo l'indice p) la condizione $w_1\mathbf{c}_1 + w_2\mathbf{c}_2 = \mathbf{x}$ dà un'equazione per ciascuna componente, cioè il sistema

$$\begin{cases} 2w_1 + w_2 = 5 \\ w_1 + 2w_2 = 5 \end{cases}$$

Sottraendo la seconda equazione dalla prima resta $w_1 - w_2 = 0$, cioè $w_1 = w_2$; sostituendo nella prima si ha $3w_1 = 5$, quindi $w_1 = w_2 = 5/3 \approx 1{,}667$. Allo stesso modo il punto $(-3,0)^\top$ ha encoding $(-2,1)^\top$: infatti $-2\mathbf{c}_1 + \mathbf{c}_2 = (-4+1, -2+2)^\top = (-3,0)^\top$. La scorciatoia delle basi ortonormali qui non funziona: $\mathbf{C}^\top\mathbf{x} = (15,15)^\top$ non è l'encoding, perché $\mathbf{c}_1$ e $\mathbf{c}_2$ non sono ortogonali ($\mathbf{c}_1^\top\mathbf{c}_2 = 4$).

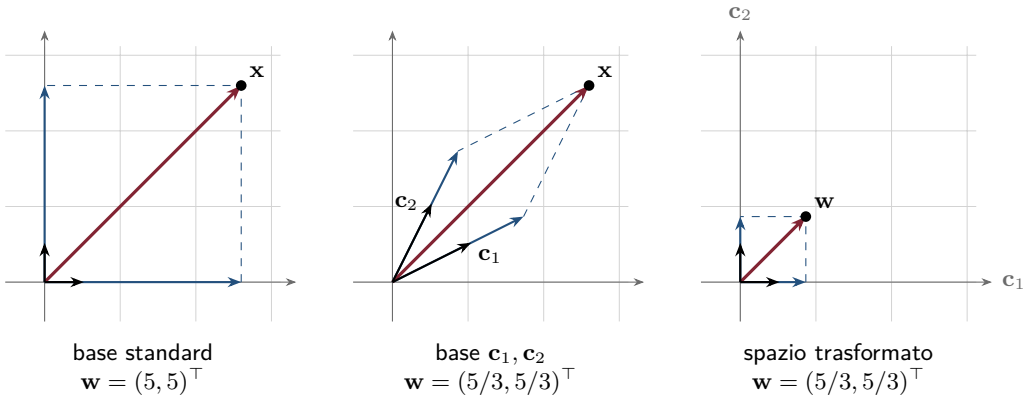


Figura 2.2. Il punto $\mathbf{x} = (5,5)^\top$ sulla base standard (a sinistra) e sulla base $\mathbf{c}_1 = (2,1)^\top$, $\mathbf{c}_2 = (1,2)^\top$ (al centro): in nero i vettori della base, in blu i vettori scalati dai pesi, in rosso la loro somma, che è il punto. A destra l'encoding $\mathbf{w}$ nello spazio trasformato, i cui assi corrispondono a $\mathbf{c}_1$ e $\mathbf{c}_2$.

2.7.1 Trovare i pesi

Nell'esempio il sistema era piccolo e si risolveva a mano. In generale conviene vedere il calcolo dei pesi come un problema di ottimizzazione: si cercano i pesi con cui la combinazione dei vettori della base, $\mathbf{C}\mathbf{w}_p$, cade il più vicino possibile al punto $\mathbf{x}_p$. ($\mathbf{C}$ ha per colonne gli N vettori della base, quindi è $N \times N$.) La distanza tra due punti è la norma della loro differenza, e la si prende al quadrato, che toglie la radice senza spostare il minimo; per ogni punto si minimizza quindi il costo h, cioè questa distanza al quadrato in funzione dei pesi:

$$h(\mathbf{w}_p) = \|\mathbf{C}\mathbf{w}_p - \mathbf{x}_p\|_2^2.$$

Quando la base genera tutto lo spazio il minimo è zero, perché esiste una combinazione che riproduce esattamente il punto. Scriverlo come minimizzazione però è utile, perché lo stesso problema ha senso anche quando la riproduzione esatta è impossibile.

Il minimo si trova annullando il **gradiente** di h, il vettore delle sue derivate rispetto ai singoli pesi: indica in che direzione h cresce, e nel punto di minimo è nullo, come la derivata in fondo a una parabola. Annullarlo dà un sistema di equazioni lineari nei pesi, le **equazioni normali**:

$$\mathbf{C}^\top\mathbf{C}\,\mathbf{w}_p = \mathbf{C}^\top\mathbf{x}_p.$$

Il sistema si dice simmetrico perché lo è la sua matrice $\mathbf{C}^\top\mathbf{C}$: l'elemento (n,m), $\mathbf{c}_n^\top\mathbf{c}_m$, è uguale all'elemento (m,n). Nell'esempio, $\mathbf{C}^\top\mathbf{C} = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix}$ e $\mathbf{C}^\top\mathbf{x} = (15,15)^\top$: il sistema $5w_1 + 4w_2 = 15$, $4w_1 + 5w_2 = 15$ dà di nuovo $w_1 = w_2 = 5/3$. Con una base ortonormale, invece, $\mathbf{C}^\top\mathbf{C} = \mathbf{I}$, e si ritrova $\mathbf{w}_p = \mathbf{C}^\top\mathbf{x}_p$. Per trovare i pesi di tutti i punti insieme si somma (o si media) il costo dei singoli punti, $\frac{1}{P}\sum_p \|\mathbf{C}\mathbf{w}_p - \mathbf{x}_p\|_2^2$. Finché la base è fissata non cambia nulla, perché ogni punto ha i suoi pesi; la forma media diventerà importante nel Capitolo 3, quando anche la base andrà imparata.

2.8 Sottospazi e proiezioni

Una base rappresenta perfettamente qualunque punto di uno spazio a N dimensioni grazie a due condizioni:

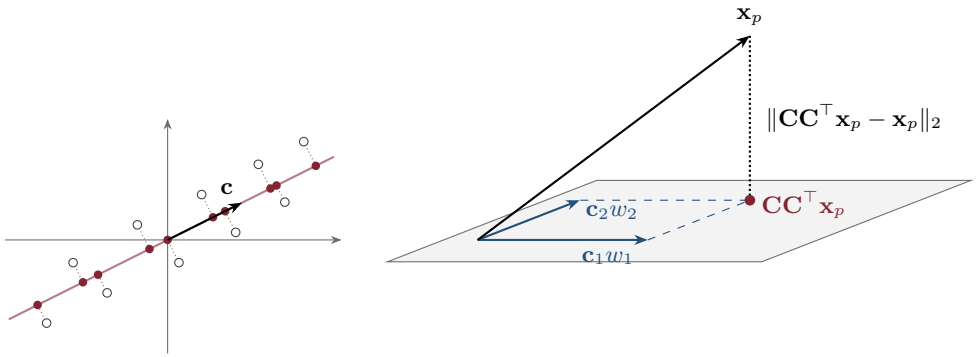
1. i vettori sono linearmente indipendenti, cioè puntano in direzioni diverse;
2. i vettori sono almeno N.

Che cosa succede se si rinuncia alla seconda, e i vettori sono solo $K < N$? Non si può più rappresentare perfettamente un insieme generico di P punti: in N dimensioni, K vettori generano al massimo un **sottospazio** di dimensione K. Per esempio, con $N = 3$:

- $K = 2$ vettori generano al più un piano;
- $K = 1$ vettore genera soltanto una retta.

Un sottospazio passa sempre per l'origine: anche per questo conviene lavorare con dati centrati.

Se il punto non sta nel sottospazio non lo si può riprodurre esattamente, ma ci si può avvicinare il più possibile: è proprio il problema di minimo della sezione precedente, ora con un minimo diverso da zero. I pesi si trovano quindi con le stesse equazioni normali, $\mathbf{C}^\top \mathbf{C}\, \mathbf{w}_p = \mathbf{C}^\top \mathbf{x}_p$, con una sola differenza: ora $\mathbf{C}$ ha dimensione $N \times K$, perché contiene solo i K vettori della base, e quindi $\mathbf{w}_p \in \mathbb{R}^K$.



$N = 2$, $K = 1$: proiezione su una retta — $N = 3$, $K = 2$: proiezione su un piano

Figura 2.3. A sinistra, punti di $\mathbb{R}^2$ (cerchi vuoti) proiettati sulla retta generata da un solo vettore $\mathbf{c}$: i punti pieni sono le proiezioni. A destra, un punto $\mathbf{x}_p$ di $\mathbb{R}^3$ proiettato sul piano generato da due vettori ortonormali $\mathbf{c}_1$ e $\mathbf{c}_2$: il punto proiettato è $\mathbf{C}\mathbf{C}^\top \mathbf{x}_p = \mathbf{c}_1 w_1 + \mathbf{c}_2 w_2$, e il segmento punteggiato è l'errore di approssimazione.

Questo procedimento si chiama **proiezione**, perché il punto più vicino del sottospazio è quello che si ottiene «facendo cadere» il punto perpendicolarmente sul sottospazio (Figura 2.3): qualunque altro punto del sottospazio è più lontano, come in un triangolo rettangolo l'ipotenusa è più lunga del cateto. Il vettore dei pesi $\mathbf{w}_p$ è l'encoding del punto nel sottospazio: K numeri invece di N. Il passo di **decoding** ricostruisce il punto combinando l'encoding con i K vettori della base: il risultato $\mathbf{C}\mathbf{w}_p$ è il **punto proiettato**, la migliore approssimazione del punto originale all'interno del sottospazio (Figura 2.4).

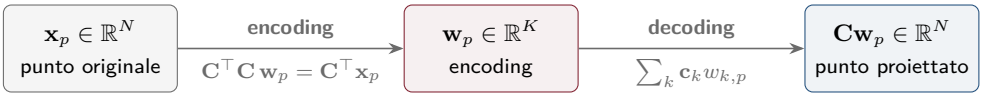


Figura 2.4. Encoding e decoding con un sottospazio di dimensione K.

Se la base di K elementi è ortonormale, l'encoding è di nuovo fatto di prodotti scalari, $\mathbf{w}_p = \mathbf{C}^\top \mathbf{x}_p$, e il decoding dà

$$\mathbf{C}\mathbf{C}^\top \mathbf{x}_p \approx \mathbf{x}_p.$$

Al posto dell'uguaglianza della Sezione 2.6 ora c'è un'approssimazione: il punto proiettato coincide con $\mathbf{x}_p$ solo se $\mathbf{x}_p$ sta già nel sottospazio. Il motivo è che nel prodotto conta l'ordine. $\mathbf{C}^\top \mathbf{C}$ è ancora l'identità: è la tabella $K \times K$ dei prodotti scalari tra i vettori della base. $\mathbf{C}\mathbf{C}^\top$ invece è una matrice $N \times N$, e non è più l'identità: è la matrice che proietta sul sottospazio. L'errore commesso è la distanza tra il punto e la sua proiezione, $\|\mathbf{C}\mathbf{C}^\top \mathbf{x}_p - \mathbf{x}_p\|_2$.

Esempio (proiezione su una retta). Si riprenda il punto $\mathbf{x} = (3,1)^\top$ dell'esempio sugli assi ruotati, ma tenendo solo il primo vettore, $\mathbf{c}_1 = (1,1)^\top/\sqrt{2}$: ora $K = 1$, $N = 2$, e il sottospazio è la retta $y = x$. L'encoding è un solo numero, $w_1 = \mathbf{c}_1^\top \mathbf{x} = 2\sqrt{2}$. Il decoding dà il punto proiettato $w_1 \mathbf{c}_1 = (2,2)^\top$, il punto della retta più vicino a $(3,1)$. In forma di matrice $\mathbf{C}$ è la sola colonna $\mathbf{c}_1$: $\mathbf{C}^\top \mathbf{C} = \mathbf{c}_1^\top \mathbf{c}_1 = 1$, mentre $\mathbf{C}\mathbf{C}^\top = \mathbf{c}_1 \mathbf{c}_1^\top$, una colonna per una riga, è la matrice $\frac{1}{2}\left[\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right]$, che manda $(3,1)^\top$ proprio in $\frac{1}{2}(4,4)^\top = (2,2)^\top$. L'errore è la distanza tra i due, $\|(3,1) - (2,2)\|_2 = \|(1,-1)\|_2 = \sqrt{2}$, e non è un caso che sia uguale (in valore assoluto) al peso $w_2 = -\sqrt{2}$ che si è buttato via: togliere un asse di una base ortonormale significa perdere esattamente la parte del punto lungo quell'asse.

Capitolo 3

Principal Component Analysis

Nel Capitolo 2 la base era sempre data, e si trattava solo di trovare i pesi con cui rappresentare i punti. Qui si fa il passo successivo: si impara anche la base, scegliendo le direzioni che descrivono meglio i dati. Il risultato è la **Principal Component Analysis** (PCA, analisi delle componenti principali), il metodo più importante di unsupervised learning per rappresentare i dati con meno numeri.

3.1 Le trasformazioni lineari

Una matrice $\mathbf{A}$ di dimensione $m \times n$ (m righe e n colonne) trasforma ogni vettore $\mathbf{x} \in \mathbb{R}^n$, fatto di n numeri, in un vettore $\mathbf{Ax} \in \mathbb{R}^m$, fatto di m numeri. Conviene pensarla come una trasformazione dell'intero spazio, e per capire che cosa fa basta guardare dove manda i vettori della base standard. Le colonne di $\mathbf{A}$ sono proprio le immagini di $\mathbf{e}_1, \mathbf{e}_2, \ldots$: letto per colonne, $\mathbf{Ae}_1$ prende una volta la prima colonna e zero volte le altre. Ogni altro vettore, che è una combinazione dei $\mathbf{e}_n$, finisce nella stessa combinazione delle colonne. Con

$$\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix},$$

per esempio, $\mathbf{e}_1$ va in $(2,0)^\top$, $\mathbf{e}_2$ resta $(0,1)^\top$, e $(1,1)^\top = \mathbf{e}_1 + \mathbf{e}_2$ va nella somma delle due colonne, $(2,1)^\top$: il piano si allunga del doppio in orizzontale. A seconda della struttura di $\mathbf{A}$ l'effetto è diverso (Figura 3.1):

- **scala**: se $\mathbf{A}$ è diagonale, cioè ha zeri fuori dalla diagonale, ogni asse viene allungato o accorciato del fattore che sta sulla diagonale, come nell'esempio;
- **rotazione** o **riflessione**: se $\mathbf{A}$ è ortonormale ($\mathbf{A}^\top\mathbf{A} = \mathbf{I}$), lo spazio ruota o si specchia senza deformarsi, e lunghezze e angoli restano gli stessi;
- **shear** (deformazione di taglio): gli elementi fuori dalla diagonale inclinano le direzioni, e un quadrato diventa un parallelogramma;
- **proiezione**: se il rango di $\mathbf{A}$ (la dimensione dello spazio generato dalle sue colonne) è $k < n$, lo spazio viene schiacciato su un sottospazio di dimensione k. Punti diversi possono finire nello stesso punto, quindi una parte dell'informazione si perde.

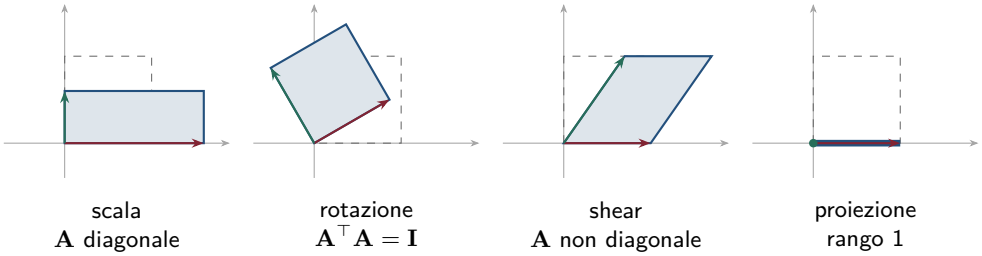


Figura 3.1. Il quadrato unitario (tratteggiato) e la sua immagine sotto quattro trasformazioni lineari. Le frecce sono le immagini di $\mathbf{e}_1$ (in rosso) e di $\mathbf{e}_2$ (in verde), cioè le colonne di $\mathbf{A}$; nella proiezione $\mathbf{e}_2$ finisce nell'origine.

Nella PCA contano soprattutto due di questi casi: le rotazioni, che cambiano gli assi senza cambiare i dati, e le proiezioni, che riducono le dimensioni.

3.2 Encoding, decoding ed errore di ricostruzione

Si riprendano le basi ortonormali della Sezione 2.6, con i vettori della base nelle colonne della matrice $\mathbf{C}$. Se i vettori sono N, tanti quante le dimensioni, l'encoding $\mathbf{w}_p = \mathbf{C}^\top \mathbf{x}_p$ è una rotazione, che cambia solo gli assi lungo cui si misura il punto, e il decoding $\mathbf{C}\mathbf{w}_p$ lo restituisce esattamente. Se sono $K < N$, il decoding $\mathbf{C}\mathbf{C}^\top \mathbf{x}_p$ è solo la proiezione del punto sul sottospazio generato dalla base (Sezione 2.8), e ciò che il punto ha fuori dal sottospazio si perde.

Quanto si perde lo misura l'**errore di ricostruzione**. Per ogni punto si vuole che la ricostruzione $\mathbf{C}\mathbf{w}_p$ sia il più vicino possibile al punto. La distanza tra i due è la lunghezza $\|\cdot\|_2$ della loro differenza, e sull'intero dataset, fatto di P punti, si prende la media dei quadrati delle distanze:

$$\frac{1}{P}\sum_{p=1}^{P} \|\mathbf{C}\mathbf{w}_p - \mathbf{x}_p\|_2^2.$$

Con una base ortonormale i pesi migliori sono $\mathbf{w}_p = \mathbf{C}^\top \mathbf{x}_p$, e sostituendoli l'errore dipende solo dalla base:

$$g(\mathbf{C}) = \frac{1}{P}\sum_{p=1}^{P} \|\mathbf{C}\mathbf{C}^\top \mathbf{x}_p - \mathbf{x}_p\|_2^2.$$

Ogni addendo è il quadrato della distanza perpendicolare tra il punto e il sottospazio (il segmento punteggiato della Figura 2.3). Una volta fissata la base, questo errore non si può ridurre: i pesi sono già i migliori possibili. L'unico modo di abbassarlo è cambiare il sottospazio.

3.3 Imparare la base

La PCA cerca, tra tutti i sottospazi di dimensione K, quello che passa più vicino ai dati, cioè che rende minimo l'errore di ricostruzione. Si prendano i 150 fiori di IRIS, ciascuno descritto da $N = 4$ misure: la domanda è se si possa descrivere ogni fiore

con meno numeri, perdendo poco. In due dimensioni con $K = 1$ la stessa domanda si può disegnare: quale retta per l'origine sta più vicina a una nuvola di punti? A occhio è la retta lungo cui la nuvola è allungata.

Il problema da risolvere è

$$g(\mathbf{w}_1, \ldots, \mathbf{w}_P, \mathbf{C}) = \frac{1}{P} \sum_{p=1}^{P} \|\mathbf{C}\mathbf{w}_p - \mathbf{x}_p\|_2^2,$$

da minimizzare sia nei pesi sia nella base. Questa funzione di costo si chiama **autoencoder** (lineare), perché si imparano insieme l'encoding, i pesi $\mathbf{w}_p$, e il decoding, la ricostruzione $\mathbf{C}\mathbf{w}_p$: il sistema impara a ricostruire il proprio input facendolo passare per una «strettoia» di soli K numeri.

Se $\mathbf{C}$ è ortonormale, i pesi ottimi sono $\mathbf{C}^\top \mathbf{x}_p$ e il costo si riduce a $g(\mathbf{C})$ della sezione precedente. Non serve nemmeno imporre l'ortonormalità come vincolo: ciò che conta per la ricostruzione è il sottospazio generato dalla base, e il sottospazio migliore si può sempre descrivere con una base ortonormale. Da qui in avanti, quindi, la base si considera ortonormale.

3.4 Dalla ricostruzione alla varianza

Il punto $\mathbf{x}_p$, la sua proiezione $\mathbf{C}\mathbf{C}^\top \mathbf{x}_p$ e l'errore formano un triangolo rettangolo (Figura 2.3): il punto è l'ipotenusa, la proiezione e l'errore sono i cateti. Per il teorema di Pitagora, quindi,

$$\|\mathbf{C}\mathbf{C}^\top \mathbf{x}_p - \mathbf{x}_p\|_2^2 = \|\mathbf{x}_p\|_2^2 - \|\mathbf{C}^\top \mathbf{x}_p\|_2^2.$$

(La lunghezza della proiezione è uguale a quella dell'encoding, $\|\mathbf{C}^\top \mathbf{x}_p\|_2$, perché $\mathbf{C}$ ha colonne ortonormali e non cambia le lunghezze.) Con il punto $(3, 1)^\top$ proiettato sulla retta $y = x$, come nella Sezione 2.8: $\|\mathbf{x}\|_2^2 = 9 + 1 = 10$, l'encoding $2\sqrt{2}$ ha quadrato 8, e l'errore $(1, -1)^\top$ ha quadrato $2 = 10 - 8$. La lunghezza del punto non dipende dalla base, quindi *minimizzare l'errore equivale a massimizzare la lunghezza della proiezione*: la base migliore è quella su cui i punti proiettano l'«ombra» più lunga.

Le componenti dell'encoding sono i prodotti scalari $\mathbf{c}_n^\top \mathbf{x}_p$, e la sua norma al quadrato è la somma dei loro quadrati, $\|\mathbf{C}^\top \mathbf{x}_p\|_2^2 = \sum_{n=1}^{K} (\mathbf{c}_n^\top \mathbf{x}_p)^2$. Facendo la media sui punti, il problema diventa

$$\text{massimizzare} \quad \frac{1}{P} \sum_{p=1}^{P} \sum_{n=1}^{K} (\mathbf{c}_n^\top \mathbf{x}_p)^2,$$

cioè la media, su tutti i punti, della somma dei quadrati delle loro «ombre» sui K vettori della base. Non ci sono termini che mescolano vettori diversi della base: ogni $\mathbf{c}_n$ dà il suo contributo separatamente, e i vettori si possono scegliere uno alla volta (restando perpendicolari tra loro).

Per il primo vettore la quantità da massimizzare è

$$h(\mathbf{c}_1) = \frac{1}{P} \sum_{p=1}^{P} (\mathbf{c}_1^\top \mathbf{x}_p)^2.$$

Il numero $\mathbf{c}_1^\top \mathbf{x}_p$ è la coordinata del punto lungo la direzione $\mathbf{c}_1$, che ha lunghezza 1. Poiché i dati sono centrati, queste coordinate hanno media zero, e la media dei loro quadrati è proprio la loro **varianza**: $h(\mathbf{c}_1)$ è la varianza dei dati lungo la direzione $\mathbf{c}_1$, cioè quanto la nuvola si allarga in quella direzione. Il primo vettore della base va quindi scelto lungo la direzione in cui i dati variano di più, come suggeriva l'intuizione: in IRIS, quella in cui i 150 fiori differiscono di più tra loro.

3.5 La matrice di covarianza

Per trovare quella direzione conviene raccogliere i dati in una matrice $\mathbf{X}$, di dimensione $N \times P$, che ha per colonne i punti $\mathbf{x}_1, \ldots, \mathbf{x}_P$ (al contrario della tabella dei dati, che ha un punto per riga). La varianza lungo $\mathbf{c}_1$ si riscrive allora come

$$h(\mathbf{c}_1) = \frac{1}{P}\, \mathbf{c}_1^\top \mathbf{X}\mathbf{X}^\top \mathbf{c}_1 = \mathbf{c}_1^\top \Sigma\, \mathbf{c}_1, \qquad \text{con} \quad \Sigma = \frac{1}{P}\mathbf{X}\mathbf{X}^\top = \frac{1}{P}\sum_{p=1}^{P} \mathbf{x}_p \mathbf{x}_p^\top.$$

L'ultima forma, la più comoda nei conti a mano, è la media dei **prodotti esterni** $\mathbf{x}_p \mathbf{x}_p^\top$ dei singoli punti: una colonna per una riga, cioè la matrice $N \times N$ con i prodotti di ogni componente del punto per ogni altra. Con $\mathbf{x}_p = (1, 2)^\top$, per esempio, $\mathbf{x}_p \mathbf{x}_p^\top = \left[\begin{smallmatrix} 1 & 2 \\ 2 & 4 \end{smallmatrix}\right]$.

La matrice Σ (sigma maiuscola), di dimensione $N \times N$, è la **matrice di covarianza** empirica, cioè calcolata dai dati, e riassume come variano le feature, da sole e a coppie:

- sulla diagonale, Σ_{ii} è la varianza della feature i: quanto quella feature si allarga attorno alla sua media;
- fuori dalla diagonale, Σ_{ij} è la **covarianza** tra le feature i e j: è positiva se le due tendono a crescere insieme, negativa se quando una cresce l'altra tende a calare, vicina a zero se non c'è una relazione lineare tra le due.

In IRIS, per esempio, la lunghezza e la larghezza del petalo hanno una covarianza positiva: i petali lunghi tendono anche a essere larghi.

3.5.1 Autovettori e autovalori

Geometricamente, Σ descrive la forma della nuvola di punti: in quali direzioni è orientata e quanto è allungata in ciascuna, come un'ellisse (in più dimensioni, un ellissoide) adattata ai dati (Figura 3.2). Le direzioni degli assi di questa ellisse sono gli **autovettori** di Σ, cioè i vettori $\mathbf{v}$ che Σ si limita ad allungare, senza cambiarne la direzione:

$$\Sigma \mathbf{v}_n = \lambda_n \mathbf{v}_n.$$

Il fattore λ_n (lambda) è l'**autovalore** corrispondente, e dice quanta varianza hanno i dati lungo $\mathbf{v}_n$. Per esempio, la matrice $\left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$ manda $(1, 1)^\top$ in $(3, 3)^\top = 3\,(1, 1)^\top$: $(1, 1)^\top$ è un autovettore, con autovalore 3. Invece $(1, 0)^\top$ finisce in $(2, 1)^\top$, che punta in un'altra direzione, quindi non è un autovettore.

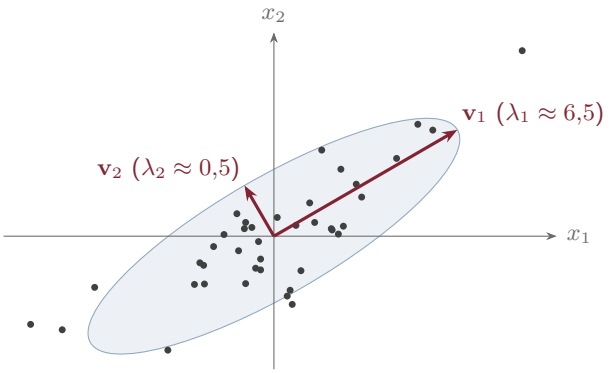


Figura 3.2. La matrice di covarianza come forma della nuvola di punti. Gli autovettori $\mathbf{v}_1$ e $\mathbf{v}_2$ indicano gli assi dell'ellisse; le frecce sono lunghe quanto i semiassi, cioè due deviazioni standard ($2\sqrt{\lambda_n}$) lungo ciascuna direzione.

La covarianza è una matrice simmetrica ($\Sigma_{ij} = \Sigma_{ji}$), e le matrici simmetriche reali hanno due proprietà che qui fanno tutto il lavoro:

- gli autovalori sono reali, e per una matrice di covarianza anche non negativi (una varianza non può essere negativa);
- gli autovettori di autovalori diversi sono ortogonali, e si possono prendere di lunghezza 1: formano quindi una base ortonormale, cioè un nuovo sistema di assi cartesiani, ruotato rispetto a quello di partenza.

Nel sistema di assi degli autovettori la trasformazione Σ è semplicissima: allunga ciascun asse $\mathbf{v}_n$ del fattore λ_n e basta. È ciò che dice la **decomposizione spettrale**

$$\Sigma = \mathbf{V}\Lambda\mathbf{V}^\top,$$

dove $\mathbf{V} = [\mathbf{v}_1 \ \cdots \ \mathbf{v}_N]$ ha gli autovettori in colonna (ed è ortonormale, $\mathbf{V}^\top\mathbf{V} = \mathbf{I}$) e Λ è la matrice diagonale degli autovalori. Letta da destra a sinistra: $\mathbf{V}^\top$ porta un vettore nel sistema degli autovettori (è un encoding), Λ scala ogni coordinata del suo autovalore, $\mathbf{V}$ riporta il risultato negli assi originali.

3.6 Le componenti principali

Resta da trovare il vettore $\mathbf{c}_1$, di lunghezza 1, che rende massima la varianza $\mathbf{c}_1^\top \Sigma\, \mathbf{c}_1$. La risposta è l'autovettore $\mathbf{v}_1$ con l'autovalore più grande, e il massimo vale λ_1:

$$\mathbf{c}_1 = \mathbf{v}_1, \qquad h(\mathbf{v}_1) = \lambda_1.$$

È l'asse più lungo dell'ellisse della Figura 3.2, la direzione in cui la nuvola è più allungata. Questa direzione è la **prima componente principale** del dataset.

Il secondo vettore va cercato allo stesso modo, $h(\mathbf{c}_2) = \mathbf{c}_2^\top \Sigma\, \mathbf{c}_2$, ma perpendicolare al primo, $\mathbf{c}_1^\top \mathbf{c}_2 = 0$. Tra le direzioni perpendicolari a $\mathbf{v}_1$ il massimo è il secondo autovalore, λ_2, raggiunto per $\mathbf{c}_2 = \mathbf{v}_2$, la **seconda componente principale**. Si procede così fino a K.

I primi K autovettori della matrice di covarianza sono le **componenti principali**, e ciascun autovalore dice quanta varianza cattura la sua componente. Poiché

massimizzare la varianza equivale a minimizzare l'errore di ricostruzione, la PCA classica è proprio il minimo globale dell'autoencoder lineare. La Figura 3.3 mostra il risultato su un dataset in due dimensioni con $K = 1$.

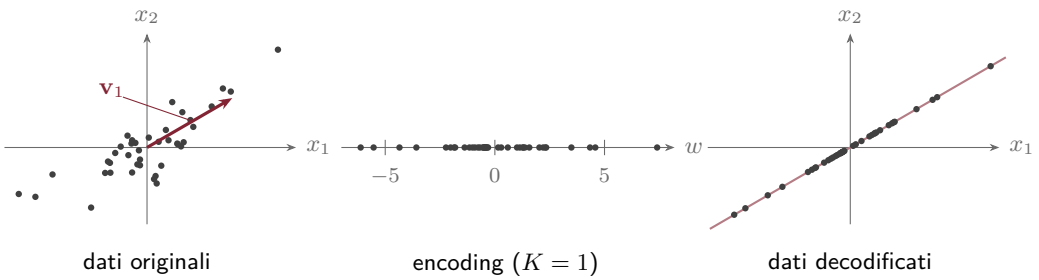


Figura 3.3. PCA con $K = 1$ sul dataset della Figura 3.2. A sinistra i dati e la prima componente principale $\mathbf{v}_1$; al centro l'encoding, un solo numero per punto ($w_p = \mathbf{v}_1^\top \mathbf{x}_p$); a destra i punti decodificati, $w_p \mathbf{v}_1$, che stanno tutti sulla retta di $\mathbf{v}_1$.

La varianza totale dei dati, la somma delle varianze delle singole feature, è uguale alla somma di tutti gli autovalori. Tenendo le prime K componenti, la frazione di varianza conservata, la **varianza spiegata**, è

$$\frac{\lambda_1 + \cdots + \lambda_K}{\lambda_1 + \cdots + \lambda_N},$$

e per Pitagora l'errore di ricostruzione medio è proprio la varianza scartata, $\lambda_{K+1} + \cdots + \lambda_N$. Nel dataset della figura $\lambda_1 \approx 6{,}51$ e $\lambda_2 \approx 0{,}50$: la prima componente da sola spiega circa il 93% della varianza. In IRIS gli autovalori sono circa 4,20; 0,24; 0,08 e 0,02: la prima componente spiega il 92% della varianza, e le prime due insieme il 98%.

3.7 A che cosa serve

- **Riduzione della dimensionalità**. Tenendo solo le prime $K < N$ componenti principali si passa da N a K numeri per punto, comprimendo i dati ma conservando quanta più variabilità (informazione) possibile. In IRIS, con $K = 2$, ogni fiore passa da quattro numeri a due, e si perde solo il 2% della varianza.
- **Feature engineering** (costruzione di feature nuove). Le componenti principali sono feature nuove e *scorrelate*: nel sistema degli autovettori la covarianza è la matrice diagonale Λ, quindi le covarianze tra feature diverse sono zero. Usarle al posto delle feature originali può semplificare il lavoro dei modelli successivi, migliorarne le prestazioni e ridurre l'*overfitting*, cioè il rischio che un modello si adatti troppo ai dati di addestramento e sbagli su quelli nuovi.
- **Unsupervised learning**. La PCA non usa etichette: impara la struttura direttamente dai dati. In IRIS la specie non entra nel calcolo, eppure la prima componente separa già tutte le setosa dalle altre due specie. È il metodo

fondamentale di *representation learning* (apprendimento di una rappresentazione dei dati) non supervisionato: rivela le direzioni di forte correlazione e le ridondanze (feature che dicono quasi la stessa cosa).

3.8 Un esempio svolto

Esempio (PCA a mano su tre punti). Si considerino i tre punti di $\mathbb{R}^2$, non centrati,

$$\mathbf{x}_1 = (2,1)^\top, \qquad \mathbf{x}_2 = (0,0)^\top, \qquad \mathbf{x}_3 = (1,-1)^\top.$$

1. *Centratura.* La media è $\bar{\mathbf{x}} = \frac{1}{3}((2,1)^\top + (0,0)^\top + (1,-1)^\top) = (1,0)^\top$. Sottraendola si ottengono i punti centrati, con la tilde per distinguerli da quelli di partenza: $\tilde{\mathbf{x}}_1 = (1,1)^\top$, $\tilde{\mathbf{x}}_2 = (-1,0)^\top$, $\tilde{\mathbf{x}}_3 = (0,-1)^\top$.

2. *Covarianza.* Con $P = 3$, $\Sigma = \frac{1}{3}\sum_p \tilde{\mathbf{x}}_p \tilde{\mathbf{x}}_p^\top$. I tre prodotti esterni sono

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \qquad \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix},$$

   la loro somma è $\left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$, e quindi $\Sigma = \frac{1}{3}\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$. Sulla diagonale le due varianze, entrambe $2/3$; fuori la covarianza, $1/3$.

3. *Autovalori.* Un autovettore soddisfa $\Sigma\mathbf{v} = \lambda\mathbf{v}$, cioè $(\Sigma - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$: la matrice $\Sigma - \lambda\mathbf{I}$ deve mandare in zero un vettore non nullo. Succede solo se schiaccia il piano su una retta, cioè se il suo **determinante** è zero. Il determinante di una matrice 2×2 $\left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$ è $ad - bc$, e vale zero proprio quando le due colonne sono allineate. Qui

$$\det(\Sigma - \lambda\mathbf{I}) = \Big(\frac{2}{3} - \lambda\Big)^2 - \frac{1}{9} = 0,$$

   quindi $\frac{2}{3} - \lambda = \pm\frac{1}{3}$, e gli autovalori sono $\lambda_1 = 1$ e $\lambda_2 = \frac{1}{3}$.

4. *Autovettori.* Per $\lambda_1 = 1$ si risolve $(\Sigma - \mathbf{I})\mathbf{v} = \mathbf{0}$. La prima equazione è $-\frac{1}{3}v_1 + \frac{1}{3}v_2 = 0$, cioè $v_1 = v_2$, e la seconda dice la stessa cosa: le soluzioni non sono un punto solo ma un'intera retta, perché il determinante è zero. Si sceglie il vettore di lunghezza 1 su quella retta, $\mathbf{v}_1 = (1,1)^\top/\sqrt{2}$. Allo stesso modo, per $\lambda_2 = \frac{1}{3}$ si trova $v_1 = -v_2$ e $\mathbf{v}_2 = (1,-1)^\top/\sqrt{2}$. La verifica: $\Sigma(1,1)^\top = \frac{1}{3}(3,3)^\top = (1,1)^\top$ e $\Sigma(1,-1)^\top = \frac{1}{3}(1,-1)^\top$. La decomposizione spettrale ha quindi $\mathbf{V} = \frac{1}{\sqrt{2}}\left[\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right]$ e $\Lambda = \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1/3 \end{smallmatrix}\right]$. Si può seguire il vettore $(1,0)^\top$ da destra a sinistra: $\mathbf{V}^\top$ lo porta in $\frac{1}{\sqrt{2}}(1,1)^\top$, Λ scala le coordinate in $\frac{1}{\sqrt{2}}(1,\frac{1}{3})^\top$, e $\mathbf{V}$ riporta il risultato negli assi originali, $(\frac{2}{3},\frac{1}{3})^\top$, che è proprio $\Sigma(1,0)^\top$.

5. *Componenti principali.* La prima è $\mathbf{v}_1$, la direzione della bisettrice; la seconda è $\mathbf{v}_2$, perpendicolare.

6. *Encoding.* Proiettando i punti centrati su $\mathbf{v}_1$ si ottengono i pesi $w_p = \mathbf{v}_1^\top \tilde{\mathbf{x}}_p$, detti anche *score*: $w_1 = \sqrt{2}$, $w_2 = -1/\sqrt{2}$, $w_3 = -1/\sqrt{2}$.
7. *Ricostruzione ed errore.* I punti ricostruiti sono $w_p \mathbf{v}_1$: $(1, 1)$, che coincide con $\tilde{\mathbf{x}}_1$, poi $(-\frac{1}{2}, -\frac{1}{2})$ due volte. Gli errori quadratici sono 0, $\frac{1}{2}$ e $\frac{1}{2}$, e la loro media è $1/3$: esattamente λ_2, la varianza scartata.
8. *Varianza spiegata.* La prima componente spiega $\lambda_1/(\lambda_1 + \lambda_2) = 1/(4/3) = 75\%$ della varianza.
9. *Segno della covarianza.* La covarianza tra le due coordinate è positiva: quando la prima è sopra la media, anche la seconda tende a esserlo. Per questo la direzione di massima varianza è la bisettrice $(1, 1)$, e non $(1, -1)$.

3.9 Quando funziona: i manifold

Dati ad alta dimensione si possono spesso rappresentare con molte meno dimensioni. Succede quando stanno vicino a una *varietà lineare* dello spazio, cioè a un sottospazio: se lo si trova, si possono proiettare i dati su di esso e descriverli con le sue coordinate senza perdere molta informazione. È esattamente ciò che fa la PCA.

Un **manifold** (varietà) è uno spazio topologico, cioè un insieme di punti in cui ha senso dire quali sono vicini tra loro, che visto da vicino assomiglia allo spazio euclideo (una retta, un piano), anche se nel complesso può essere curvo. Succede con la superficie della Terra: è una sfera, ma un campo da calcio sembra piatto. L'esempio tipico in due dimensioni è la circonferenza unitaria $x^2 + y^2 = 1$: nel complesso è una curva chiusa, ma un suo pezzo piccolo, visto da vicino, sembra un segmento.

Si prenda la metà superiore della circonferenza, i punti con $y > 0$. Ognuno è identificato in modo unico dalla sua coordinata x: la funzione $(x, y) \mapsto x$, che a ogni punto associa la sua x, manda l'arco sull'intervallo aperto $(-1, 1)$ in modo liscio e invertibile, perché dalla x si ricava il punto, $(x, \sqrt{1 - x^2})$. Le funzioni che mettono in corrispondenza biunivoca (un punto per ogni valore, e viceversa) una regione aperta della superficie con un sottoinsieme dello spazio euclideo si chiamano **chart** (carte), come le carte geografiche, che riportano in piano un pezzo della superficie terrestre. E come una sola carta non basta per tutto il globo, una sola chart non basta per tutta la circonferenza: la x non distingue un punto in alto da quello in basso. Quattro chart, sugli archi superiore, inferiore, sinistro e destro, la coprono invece tutta, come le pagine di un atlante che si sovrappongono ai bordi (Figura 3.4).

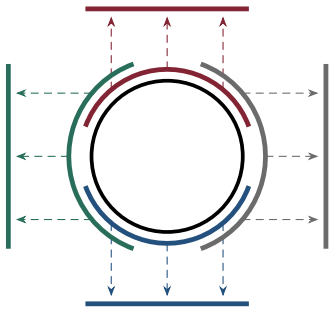


Figura 3.4. La circonferenza coperta da quattro chart: ogni arco aperto si proietta su un segmento, quelli in alto e in basso con la coordinata x, quelli a sinistra e a destra con la y. Gli archi si sovrappongono agli estremi.

Una chart si può vedere anche nel verso opposto, come funzione $\phi\colon \mathbb{R}^1 \to S \subset \mathbb{R}^2$ che dal parametro porta al punto della curva: la scrittura si legge «ϕ va dalla retta dei numeri $\mathbb{R}^1$ alla curva S, che sta nel piano $\mathbb{R}^2$». Per l'arco superiore, per esempio, $\phi(t) = (t, \sqrt{1-t^2})$. La funzione deve essere liscia e invertibile, con inversa anch'essa liscia (un **diffeomorfismo**: funzione e inversa sono derivabili con derivate continue). Il dominio di ϕ, l'insieme dei suoi input, è lo **spazio dei parametri**, ed è euclideo; la sua immagine, l'insieme dei punti che produce, è l'**embedding**, la curva o superficie nello spazio. Un manifold, alla fine, è un'unione di chart. Con un manifold lineare basta la PCA; se il manifold è curvo, come la circonferenza, un sottospazio non lo segue bene.

Capitolo 4

Ottimizzazione

Nei capitoli precedenti si è già cercato il minimo di una funzione di costo: i pesi con cui una base riproduce un punto (Capitolo 2) e la base che ricostruisce meglio i dati (Capitolo 3). In entrambi i casi il minimo aveva una formula, le equazioni normali o gli autovettori della covarianza; per quasi tutti i modelli di machine learning, invece, una formula non c'è. In questo capitolo $\mathbf{w}$ è il vettore delle variabili da scegliere, per esempio i parametri di un modello, e g la funzione da rendere minima; i pesi $\mathbf{w}_p$ dei capitoli precedenti ne sono un caso particolare.

4.1 Minimi e massimi

Ottimizzare una funzione $g(\mathbf{w})$ vuol dire trovare il valore di $\mathbf{w}$ che la rende più piccola possibile (minimizzarla) o più grande possibile (massimizzarla). La funzione si chiama **funzione obiettivo** (*objective function*); quando la si minimizza si chiama anche **loss function** (funzione di perdita), perché di solito misura quanto il modello sbaglia, come le funzioni di costo dei capitoli precedenti. Il valore migliore di $\mathbf{w}$ si indica con un asterisco, $\mathbf{w}^\star$.

Si parte da una sola variabile, w. La parabola $g(w) = w^2$ non è mai negativa e vale zero solo in $w = 0$: quindi $w^\star = 0$ è il suo **minimo globale**, il punto in cui la funzione vale meno che in qualunque altro,

$$g(w^\star) \leq g(w) \quad \text{per ogni } w.$$

Con la disuguaglianza rovesciata, $g(w^\star) \geq g(w)$ per ogni w, si ha un **massimo globale**. Minimi e massimi si scambiano cambiando segno alla funzione, perché capovolgere il grafico trasforma le valli in cime:

$$\max_{\mathbf{w}} \; g(\mathbf{w}) = -\min_{\mathbf{w}} \; (-g(\mathbf{w})),$$

dove sotto max e min si scrive la variabile che si fa cambiare. Per esempio $g(w) = 3 - w^2$ ha il massimo in $w = 0$, dove vale 3, e la funzione opposta, $w^2 - 3$, ha il minimo nello stesso punto, dove vale -3: il punto è lo stesso, il valore cambia segno. Per questo basta saper minimizzare, e d'ora in poi si parla quasi solo di minimi.

Il minimo globale non è sempre unico. La funzione $g(w) = \sin(2w)$ ripete all'infinito la stessa onda tra -1 e 1: ha infiniti minimi globali, tutti con valore -1,

e infiniti massimi globali, con valore 1. Tra -3 e 3 ci sono due minimi, in $w \approx -0{,}79$ e $w \approx 2{,}36$, e due massimi (Figura 4.1, al centro).

Se all'onda si somma una parabola, come in $g(w) = \sin(3w) + 0{,}1\,w^2$, le valli non stanno più tutte alla stessa altezza: la parabola solleva sempre di più quelle lontane da zero (Figura 4.1, a destra). Il minimo globale è uno solo, in $w \approx -0{,}51$, dove $g \approx -0{,}97$. Il fondo delle altre valli, come quelle in $w \approx 1{,}54$ e $w \approx -2{,}56$, è il punto più basso solo nei dintorni: è un **minimo locale**. In formula, $w^\star$ è un minimo locale se

$$g(w^\star) \le g(w) \quad \text{per ogni } w \text{ vicino a } w^\star.$$

Allo stesso modo le cime, in $w \approx -1{,}61$, $0{,}54$ e $2{,}68$, sono **massimi locali**. Un minimo globale è sempre anche locale, ma non viceversa.

Si pensi a un escursionista che vuole raggiungere il punto più basso di una zona di montagna: il fondo di ogni conca è un minimo locale, il fondo della conca più profonda è il minimo globale. Ma c'è la nebbia, e l'unica cosa che si sente è la pendenza del terreno sotto i piedi. L'ottimizzazione lavora proprio così: di una funzione complicata non si vede il grafico, e si può solo calcolare la pendenza nel punto in cui ci si trova.

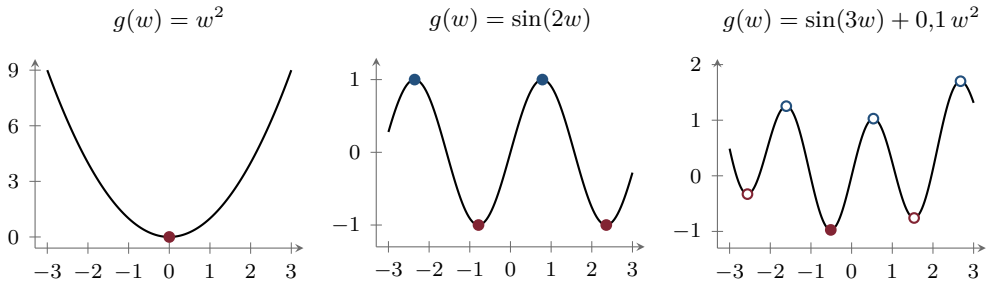


Figura 4.1. Minimi e massimi di tre funzioni tra -3 e 3: in rosso i minimi, in blu i massimi; i punti pieni sono globali, quelli vuoti solo locali.

Queste definizioni usano soltanto i valori di g, e per questo si chiamano **condizioni di ordine zero**. Ci sono poi condizioni del **primo ordine**, che usano la derivata prima di g, e del **secondo ordine**, che usano la derivata seconda.

4.2 La condizione del primo ordine

La **derivata** $g'(w)$, che si scrive anche $\frac{\mathrm{d}}{\mathrm{d}w}\,g(w)$, è la pendenza della retta tangente al grafico nel punto w: dice di quanto cresce g, per ogni unità di spostamento, quando w si sposta di pochissimo. Per $g(w) = w^2$ la derivata è $g'(w) = 2w$. In $w = 1$ vale 2: passando da 1 a 1,01, g va da 1 a 1,0201, cioè cresce di circa il doppio dello spostamento. In $w = -1$ vale -2, perché lì la funzione scende; in $w = 0$ vale 0, e la tangente è orizzontale.

Nel fondo di una valle la funzione ha smesso di scendere e non ha ancora cominciato a salire, quindi la tangente è orizzontale. È la **condizione del primo ordine**: in un minimo la derivata è nulla,

$$g'(w^\star) = 0.$$

Vale per qualunque funzione che abbia la derivata.

Il viceversa però non vale. La derivata è nulla anche nei massimi, dove la tangente è di nuovo orizzontale, e in punti che non sono né l'uno né l'altro: per $g(w) = w^3$ la derivata, $3w^2$, si annulla in $w = 0$, ma lì la funzione si appiattisce per un istante e poi riprende a salire. Un punto così si chiama **punto di sella** (*saddle point*). I punti in cui la derivata è nulla si chiamano **punti stazionari**, e sono solo i candidati tra cui cercare i minimi (Figura 4.2).

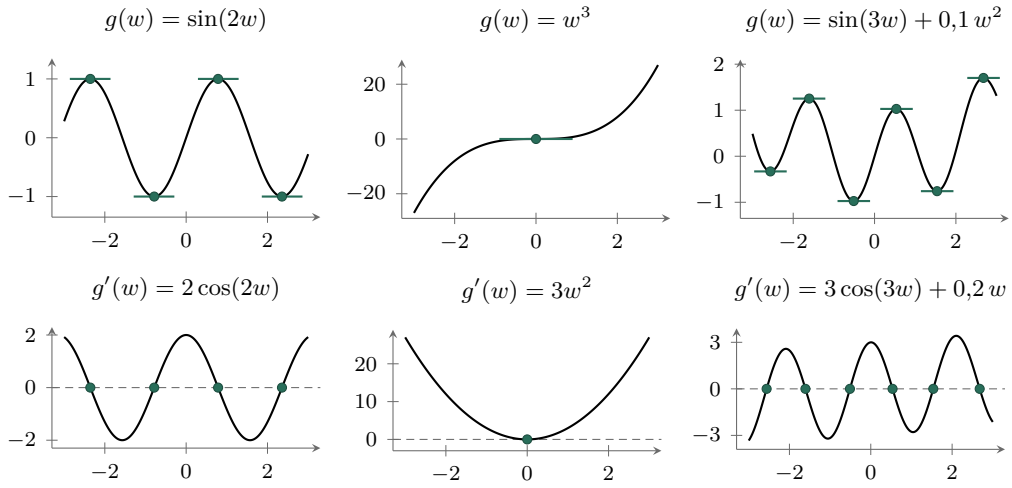


Figura 4.2. In alto tre funzioni con i loro punti stazionari e le tangenti orizzontali (in verde); in basso le loro derivate, che si annullano proprio lì.

Con più variabili, $\mathbf{w} = (w_1, \ldots, w_N)^\top$, si deriva una variabile alla volta. La **derivata parziale** $\partial g/\partial w_n$ è la derivata di g rispetto a w_n, con tutte le altre variabili tenute ferme. Per esempio $g(w_1, w_2) = w_1^2 + w_2^2 + 2$ è una «ciotola» con il fondo in $(0, 0)$, e

$$\frac{\partial g}{\partial w_1} = 2w_1, \qquad \frac{\partial g}{\partial w_2} = 2w_2 :$$

derivando rispetto a w_1, i termini w_2^2 e 2 sono costanti, e la derivata di una costante è zero. In un minimo la tangente è un piano (o un iperpiano) orizzontale, quindi tutte le derivate parziali sono nulle insieme. Con N variabili i candidati sono le soluzioni di un sistema di N equazioni, il **sistema del primo ordine**,

$$\frac{\partial g}{\partial w_1}(\mathbf{w}) = 0, \quad \ldots, \quad \frac{\partial g}{\partial w_N}(\mathbf{w}) = 0, \qquad \text{in breve} \qquad \nabla g(\mathbf{w}) = \mathbf{0}.$$

Il vettore $\nabla g(\mathbf{w})$, che si legge «nabla g», raccoglie le N derivate parziali ed è il **gradiente** di g, già incontrato nel Capitolo 2; $\mathbf{0}$ è il vettore di N zeri. Per la ciotola il sistema è $2w_1 = 0$, $2w_2 = 0$, con l'unica soluzione $(0, 0)$, dove $g = 2$.

A volte il sistema si risolve a mano, come per la ciotola, e il minimo si trova in **forma chiusa**, cioè con una formula: è il caso delle equazioni normali del Capitolo 2. Ma già per una funzione semplice come

$$g(w) = \frac{1}{50}\left(w^4 + w^2 + 10w\right)$$

la condizione del primo ordine, $\frac{1}{50}(4w^3 + 2w + 10) = 0$, cioè $2w^3 + w + 5 = 0$, è un'equazione di terzo grado, e la sua unica soluzione è

$$w = \frac{\sqrt[3]{\sqrt{2031} - 45}}{6^{2/3}} - \frac{1}{\sqrt[3]{6\left(\sqrt{2031} - 45\right)}} \approx -1{,}23.$$

Con molte variabili e funzioni più complicate di solito una formula non c'è proprio. Si usano allora **metodi iterativi**: si parte da un punto e lo si migliora un passo alla volta, nella direzione indicata dal gradiente.

4.3 Il gradiente

Che cosa dice il gradiente lo si vede bene sulle funzioni più semplici, quelle «piatte». Un **iperpiano** è una funzione della forma

$$h(\mathbf{w}) = a + b_1 w_1 + b_2 w_2 + \cdots + b_N w_N = a + \mathbf{b}^\top \mathbf{w},$$

dove a è un numero, $\mathbf{b} = (b_1, \ldots, b_N)^\top$ un vettore di coefficienti e $\mathbf{b}^\top \mathbf{w}$ il loro prodotto scalare. Con $N = 1$ è una retta, $h(w) = a + bw$, con pendenza b; con $N = 2$ è un piano inclinato sopra il piano (w_1, w_2), e così via. A differenza dei sottospazi del Capitolo 2, il grafico di un iperpiano non deve passare per l'origine: lo alza o lo abbassa a. Le derivate parziali di un iperpiano sono i suoi coefficienti, $\partial h / \partial w_n = b_n$, quindi il suo gradiente è $\mathbf{b}$, lo stesso in ogni punto.

Ci si trova in un punto $\mathbf{w}_0$ e ci si sposta in $\mathbf{w}_0 + \mathbf{d}$, dove la direzione $\mathbf{d}$ è un vettore di lunghezza 1, $\|\mathbf{d}\|_2 = 1$, che dice solo da che parte andare. Se h cresce è una **direzione di salita** (*ascent direction*), se cala una **direzione di discesa** (*descent direction*). Con $N = 1$ le direzioni sono solo due, a destra e a sinistra, e con pendenza b positiva la prima sale e la seconda scende; con $N \geq 2$ sono infinite.

Quale direzione fa salire h più di tutte? È la **direzione di massima salita** (*steepest ascent*), la soluzione di

$$\max_{\mathbf{d}} \; h(\mathbf{w}_0 + \mathbf{d}) \quad \text{con} \quad \|\mathbf{d}\|_2 = 1.$$

Sviluppando, $h(\mathbf{w}_0 + \mathbf{d}) = a + \mathbf{b}^\top \mathbf{w}_0 + \mathbf{b}^\top \mathbf{d}$. I primi due termini non dipendono da $\mathbf{d}$, quindi basta rendere massimo $\mathbf{b}^\top \mathbf{d}$, la variazione di h. Il prodotto scalare si può scrivere come $\mathbf{b}^\top \mathbf{d} = \|\mathbf{b}\|_2 \, \|\mathbf{d}\|_2 \cos\theta$, dove θ è l'angolo tra le due frecce; qui $\|\mathbf{d}\|_2 = 1$ e $\|\mathbf{b}\|_2$ non dipende da $\mathbf{d}$, quindi tutto si riduce a rendere massimo $\cos\theta$:

- $\cos\theta$ vale 1, il suo massimo, per $\theta = 0$: la direzione di massima salita è $\mathbf{d} = \mathbf{b}/\|\mathbf{b}\|_2$, quella di $\mathbf{b}$;
- vale -1, il minimo, per $\theta = 180°$: la direzione di **massima discesa** (*steepest descent*) è quella opposta, $\mathbf{d} = -\mathbf{b}/\|\mathbf{b}\|_2$;
- vale 0 per $\theta = 90°$: nelle direzioni perpendicolari a $\mathbf{b}$ il valore di h non cambia.

Con $\mathbf{b} = (3, 4)^\top$, per esempio, che è lungo $\|\mathbf{b}\|_2 = \sqrt{9 + 16} = 5$, la direzione di $\mathbf{b}$ è (0,6; 0,8) e fa salire h di $3 \cdot 0{,}6 + 4 \cdot 0{,}8 = 5$, il massimo; $(1, 0)$, a 53° da $\mathbf{b}$, lo fa salire di 3; la perpendicolare (0,8; $-0{,}6$) di $2{,}4 - 2{,}4 = 0$; la direzione opposta a $\mathbf{b}$ lo fa scendere di 5 (Figura 4.3).

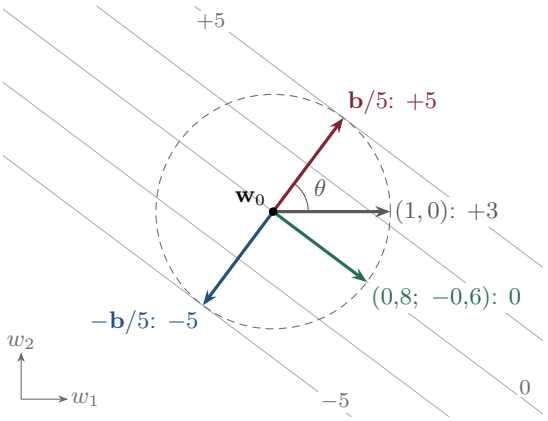


Figura 4.3. L'iperpiano $h(\mathbf{w}) = a + \mathbf{b}^\top \mathbf{w}$ con $\mathbf{b} = (3,4)^\top$, visto dall'alto nel piano (w_1, w_2). Le rette grigie sono rette di livello, su cui h è costante, e i numeri dicono di quanto h supera il suo valore in $\mathbf{w}_0$. Sul cerchio tratteggiato stanno le direzioni di lunghezza 1 attorno a $\mathbf{w}_0$, e ogni freccia arriva sulla retta del proprio valore di $\mathbf{b}^\top \mathbf{d}$.

Una funzione qualsiasi non è un iperpiano, ma vista da vicino gli somiglia, come un pezzo piccolo di curva somiglia a un segmento. Con il gradiente di g in $\mathbf{w}_0$, cioè le derivate parziali calcolate in quel punto, si costruisce l'iperpiano che tocca il grafico di g in $\mathbf{w}_0$ con la stessa pendenza in ogni direzione, l'**iperpiano tangente**:

$$h(\mathbf{w}) = g(\mathbf{w}_0) + \nabla g(\mathbf{w}_0)^\top (\mathbf{w} - \mathbf{w}_0).$$

È l'**approssimazione di Taylor del primo ordine** di g attorno a $\mathbf{w}_0$; con $N = 1$ è la retta tangente. Scritto nella forma $a + \mathbf{b}^\top \mathbf{w}$, ha per coefficienti $\mathbf{b} = \nabla g(\mathbf{w}_0)$: la sua inclinazione è il gradiente. Per $g(w) = w^2$ in $w_0 = 1$, dove $g(1) = 1$ e $g'(1) = 2$, la tangente è $h(w) = 1 + 2(w - 1) = 2w - 1$, con pendenza 2. Vicino a w_0 l'approssimazione è buona, $g(1{,}1) = 1{,}21$ e $h(1{,}1) = 1{,}20$; lontano no, $g(3) = 9$ e $h(3) = 5$.

Vicino a $\mathbf{w}_0$, quindi, g si comporta come il suo iperpiano tangente, che ha $\mathbf{b} = \nabla g(\mathbf{w}_0)$, e ne eredita le direzioni:

- la direzione di massima salita di g in $\mathbf{w}_0$ è quella del gradiente, $+\nabla g(\mathbf{w}_0)$;
- la direzione di massima discesa è quella opposta, $-\nabla g(\mathbf{w}_0)$;
- la lunghezza del gradiente, $\|\nabla g(\mathbf{w}_0)\|_2$, dice quanto rapidamente g cresce in quella direzione;
- il gradiente è perpendicolare alla **curva di livello** che passa per $\mathbf{w}_0$, la curva dei punti in cui g ha lo stesso valore che in $\mathbf{w}_0$, come le curve di una carta escursionistica uniscono i punti alla stessa quota: lungo di essa g non cambia, come nelle direzioni perpendicolari a $\mathbf{b}$.

Per la ciotola $g(w_1, w_2) = w_1^2 + w_2^2 + 2$ nel punto $\mathbf{w}_0 = (1{,}5;\ 2)$, dove $g = 8{,}25$, il gradiente è $(2w_1, 2w_2)^\top = (3,4)^\top$, lungo 5. Spostandosi di 0,01 nella direzione di massima salita, $(0{,}6;\ 0{,}8)$, g passa da 8,25 a circa 8,30, cioè cresce di $0{,}01 \cdot 5$. La direzione di massima discesa, $(-0{,}6;\ -0{,}8)$, punta dritta al fondo della ciotola, $(0, 0)$. Le curve di livello sono cerchi centrati nell'origine, e quella che passa per $\mathbf{w}_0$ ha raggio 2,5: il gradiente punta lungo il raggio, perpendicolare al cerchio.

Da qui nascono due algoritmi: il **gradient descent** (discesa del gradiente), che per arrivare a un minimo locale si muove lungo $-\nabla g$, e il **gradient ascent**, che per arrivare a un massimo locale si muove lungo $+\nabla g$. Prima, però, il gradiente bisogna saperlo calcolare.

4.4 Calcolare il gradiente

Per una funzione complicata ricavare a mano la formula della derivata è lungo e facile da sbagliare.

4.4.1 La derivata numerica

La pendenza della tangente si può approssimare con quella della retta che passa per due punti del grafico molto vicini, uno poco prima e uno poco dopo w_0. Scelto un passo piccolo ε (epsilon),

$$g'(w_0) \approx \frac{g(w_0 + \varepsilon) - g(w_0 - \varepsilon)}{2\varepsilon}.$$

Per $g(w) = w^3$ in $w_0 = 1$, con $\varepsilon = 0{,}01$, viene $(1{,}030301 - 0{,}970299)/0{,}02 = 3{,}0001$, mentre la derivata esatta, $3w^2$, vale 3. La **derivata numerica** è comoda, perché basta saper calcolare g, ma è un'approssimazione, e con più variabili va ripetuta per ogni derivata parziale.

4.4.2 Le regole di derivazione

Le derivate delle funzioni composte si calcolano a pezzi, con tre regole:

- la **regola della somma**: la derivata di una somma è la somma delle derivate;
- la **regola del prodotto** (*product rule*), per il prodotto di due funzioni: $\big(f(w)\,u(w)\big)' = f'(w)\,u(w) + f(w)\,u'(w)$;
- la **regola della catena** (*chain rule*), per una funzione dentro un'altra: $\big(f(u(w))\big)' = f'(u(w)) \cdot u'(w)$, cioè la derivata della funzione esterna, calcolata sulla funzione interna, per la derivata della funzione interna.

Per esempio, in $\sin(3w)$ la funzione esterna è il seno, che ha per derivata il coseno, e quella interna è $3w$, che ha derivata 3: la derivata è $3\cos(3w)$. Con la regola della somma, la funzione delle sezioni precedenti ha derivata $\big(\sin(3w) + 0{,}1\,w^2\big)' = 3\cos(3w) + 0{,}2\,w$.

4.4.3 Il grafo computazionale

Quasi ogni funzione scritta con una formula si scompone in **funzioni elementari** (seno, esponenziale, logaritmo, ...) combinate con poche **operazioni** (somma, prodotto, composizione). Questa scomposizione si organizza in un **grafo computazionale** (*computation graph*), che permette a un programma di calcolare la funzione un pezzo alla volta. Per esempio

$$g(w) = \tanh(w)\cos(w) + \log(w),$$

dove tanh è la tangente iperbolica, una funzione a forma di S che sale da -1 a 1, e log il logaritmo naturale, si scompone in cinque pezzi:

$$a = \tanh(w), \quad b = \cos(w), \quad c = \log(w), \quad d = a \cdot b, \quad e = c + d.$$

L'ultimo, e, è $g(w)$. Nel grafo (Figura 4.4) ogni pezzo è un nodo, e il nodo grigio è l'input w. Il nodo da cui parte una freccia è il **genitore** (*parent*) di quello a cui arriva, che ne è il **figlio** (*child*): w è genitore di a, b e c; a e b sono genitori di d, che a sua volta è genitore di e. Ogni figlio è una funzione dei suoi genitori, come $d = d(a, b)$, e quindi, risalendo, dell'input: $d(w) = \tanh(w)\cos(w)$. Il calcolo scorre di genitore in figlio. In $w = 1{,}5$:

$$a \approx 0{,}905, \quad b \approx 0{,}071, \quad c \approx 0{,}405, \quad d = a \cdot b \approx 0{,}064, \quad e = c + d \approx 0{,}469.$$

4.4.4 Forward mode

L'**automatic differentiation** (differenziazione automatica) usa lo stesso grafo per calcolare la derivata. Nel **forward mode** (modo in avanti) si percorre il grafo da sinistra a destra, come per il valore, e ogni nodo calcola due numeri: il proprio valore e la propria derivata rispetto all'input. Qui l'apice indica la derivata rispetto a w: a' è la derivata di a rispetto a w. In $w = 1{,}5$:

1. *L'input.* Il valore $w = 1{,}5$ lo sceglie chi calcola, e la derivata di w rispetto a sé stesso è $w' = 1$.
2. *I figli di w.* Servono le derivate delle funzioni elementari: quella di $\tanh(w)$ è $1 - \tanh^2(w)$, quella di $\cos(w)$ è $-\sin(w)$, quella di $\log(w)$ è $1/w$. Quindi $a' = 1 - 0{,}905^2 \approx 0{,}181$, $b' = -\sin(1{,}5) \approx -0{,}997$ e $c' = 1/1{,}5 \approx 0{,}667$.
3. *Il prodotto.* $d = a \cdot b$ dipende da w attraverso due genitori, e la regola del prodotto dà $d' = a' \cdot b + a \cdot b' \approx 0{,}181 \cdot 0{,}071 + 0{,}905 \cdot (-0{,}997) \approx 0{,}013 - 0{,}903 = -0{,}890$.
4. *La somma.* $e = c + d$, e per la regola della somma $e' = c' + d' \approx 0{,}667 - 0{,}890 = -0{,}223$. È la derivata cercata, $g'(1{,}5) \approx -0{,}223$.

In ogni passo si applica la stessa regola, che è la regola della catena sul grafo: la derivata di un figlio è la somma, su tutti i suoi genitori, della derivata parziale del figlio rispetto al genitore per la derivata del genitore. Per d le derivate parziali sono $\partial d/\partial a = b$ e $\partial d/\partial b = a$, da cui $d' = b\,a' + a\,b'$; per e sono entrambe 1, da cui $e' = d' + c'$.

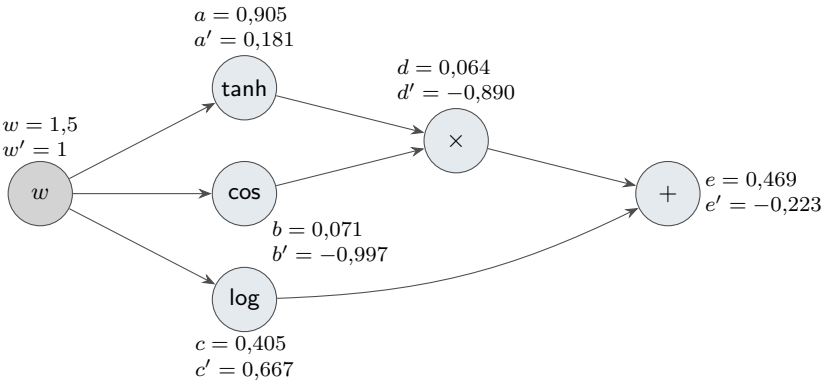


Figura 4.4. Il grafo computazionale di $g(w) = \tanh(w)\cos(w) + \log(w)$, percorso in forward mode in $w = 1{,}5$: accanto a ogni nodo il valore e la derivata rispetto a w.

Il forward mode è facile da programmare e, al contrario della derivata numerica, è esatto: la formula con $\varepsilon = 0{,}01$ darebbe $-0{,}22339$, invece di $-0{,}22343$.

4.4.5 Reverse mode e backpropagation

Con molti input il forward mode spreca calcoli, perché ogni nodo deve portarsi dietro la derivata rispetto a *ciascun* input, cioè un vettore di N numeri. Per $g(w_1, \ldots, w_4) = w_1^2 + w_2^2 + w_3^2 + w_4^2$, il nodo $a = w_1^2$ porta il vettore $(2w_1, 0, 0, 0)$: dipende solo da w_1, e tre numeri su quattro sono zeri inutili. Nella loss di una rete neurale gli input sono i parametri da ottimizzare, che possono essere milioni, e lo spreco diventa enorme.

Il **reverse mode** (modo all'indietro) lo evita con due passate sul grafo:

- nella **passata in avanti** (*forward pass*), da sinistra a destra, ogni nodo calcola il proprio valore e soltanto le derivate parziali *locali*, rispetto ai suoi genitori diretti;
- nella **passata all'indietro** (*backward pass*) si parte dall'output, con derivata 1, e si torna verso gli input, e ogni nodo riceve la derivata dell'output rispetto a sé. Lungo ogni freccia si moltiplica la derivata del figlio per la derivata parziale locale del figlio rispetto al genitore (è la regola della catena), e se un nodo ha più figli si sommano i contributi che arrivano da ciascuno.

Per $g(w_1, w_2) = w_1^2 + w_2^2$ in $(3, 1)$, con i nodi $a = w_1^2$, $b = w_2^2$ e $c = a + b$ (Figura 4.5):

1. *Passata in avanti.* $a = 9$, con derivata locale $\partial a/\partial w_1 = 2w_1 = 6$; $b = 1$, con $\partial b/\partial w_2 = 2w_2 = 2$; $c = 10$, con $\partial c/\partial a = \partial c/\partial b = 1$.
2. *Passata all'indietro.* L'output c parte con 1. Ad a arriva $1 \cdot \partial c/\partial a = 1$, e a b allo stesso modo 1. A w_1 arriva $1 \cdot \partial a/\partial w_1 = 6$, a w_2 arriva $1 \cdot 2 = 2$.

Agli input si ha tutto il gradiente in una volta: $\nabla g(3, 1) = (6, 2)^\top$, che è proprio $(2w_1, 2w_2)^\top$.

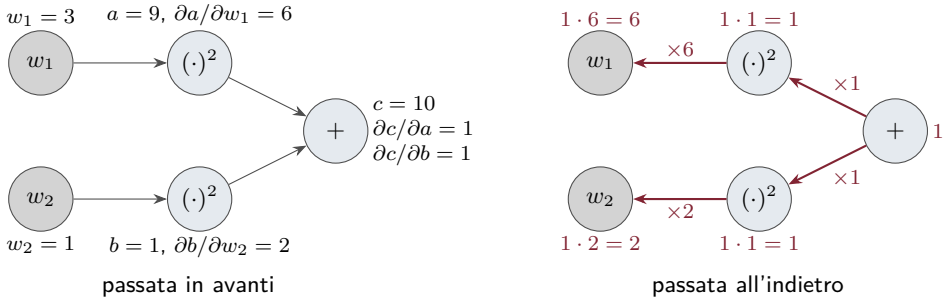


Figura 4.5. Reverse mode su $g(w_1, w_2) = w_1^2 + w_2^2$ in $(3, 1)$: a sinistra la passata in avanti, a destra quella all'indietro, in rosso, che porta agli input il gradiente $(6, 2)^\top$.

La somma dei contributi si vede nel grafo di $\tanh(w)\cos(w) + \log(w)$, dove w ha tre figli. All'indietro, d e c ricevono 1 da e; da d, il nodo a riceve $\partial d/\partial a = b \approx 0{,}071$ e il nodo b riceve $\partial d/\partial b = a \approx 0{,}905$. Infine w somma i contributi dei suoi tre figli, ciascuno dato da ciò che il figlio ha ricevuto per la sua derivata locale rispetto a w:

$$0{,}071 \cdot 0{,}181 + 0{,}905 \cdot (-0{,}997) + 1 \cdot 0{,}667 \approx 0{,}013 - 0{,}903 + 0{,}667 = -0{,}223,$$

lo stesso risultato del forward mode. La Tabella 4.1 mette a confronto questi due modi e la derivata numerica.

	Derivata numerica	Forward mode	Reverse mode
risultato	approssimato	esatto	esatto
passate sul grafo	nessuna: due valori di g per ogni input	una, in avanti	una in avanti e una all'indietro
che cosa calcola ogni nodo	–	il valore e le derivate rispetto a tutti gli N input	in avanti il valore e le derivate locali, all'indietro un numero solo
con molti input	il lavoro cresce con N	il lavoro cresce con N	tutto il gradiente con una sola passata all'indietro
memoria	poca	poca	va conservato il grafo, con i valori della passata in avanti

Tabella 4.1. Tre modi di calcolare una derivata a confronto.

Con molti input il reverse mode è quindi lo standard nel machine learning. Applicato alle reti neurali si chiama **backpropagation** (retropropagazione dell'errore): è la base di *autograd*, in Python, e di librerie come PyTorch e TensorFlow, e rende possibile addestrare reti con milioni di parametri.

4.5 Gradient descent

Il gradient descent fa come l'escursionista nella nebbia: sente da che parte il terreno scende di più, fa un passo e ripete. Parte da un punto iniziale $\mathbf{w}^{(0)}$ e a ogni passo si

sposta un po' nella direzione $-\nabla g$,

$$\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} - \alpha \, \nabla g\big(\mathbf{w}^{(k-1)}\big).$$

- $k = 1, 2, \ldots$ conta i passi, e $\mathbf{w}^{(k)}$ è il punto raggiunto dopo k passi. L'indice va tra parentesi per non confonderlo con una potenza o con le componenti $w_1, w_2, \ldots$
- $\nabla g(\mathbf{w}^{(k-1)})$ è il gradiente nel punto in cui ci si trova.
- $\alpha > 0$ è la **step size** (lunghezza del passo), detta anche **learning rate**. Lo spostamento è lungo $\alpha \, \|\nabla g\|_2$: i passi sono lunghi dove la funzione è ripida, corti dove è quasi piatta.

A ogni passo l'algoritmo approssima g con il suo iperpiano tangente nel punto corrente e scende lungo di esso (Figura 4.6). Con una step size adatta, il procedimento porta verso un minimo locale.

Esempio (gradient descent sulla ciotola). Su $g(w_1, w_2) = w_1^2 + w_2^2 + 2$, che ha gradiente $(2w_1, 2w_2)^\top$, si parte da $\mathbf{w}^{(0)} = (1{,}5;\ 2)$ con $\alpha = 0{,}2$.

1. In $\mathbf{w}^{(0)}$ il gradiente è $(3, 4)^\top$, e $\mathbf{w}^{(1)} = (1{,}5;\ 2) - 0{,}2\,(3, 4) = (0{,}9;\ 1{,}2)$.
2. In $\mathbf{w}^{(1)}$ il gradiente è $(1{,}8;\ 2{,}4)^\top$, e $\mathbf{w}^{(2)} = (0{,}9;\ 1{,}2) - 0{,}2\,(1{,}8;\ 2{,}4) = (0{,}54;\ 0{,}72)$.
3. In generale $\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} - 0{,}2 \cdot 2\,\mathbf{w}^{(k-1)} = 0{,}6\,\mathbf{w}^{(k-1)}$: ogni passo accorcia del 40% la distanza dall'origine, lungo la retta che porta al fondo.

Il costo scende da 8,25 a 4,25, poi a 2,81, 2,29, 2,10, …, e si avvicina a 2, il minimo.

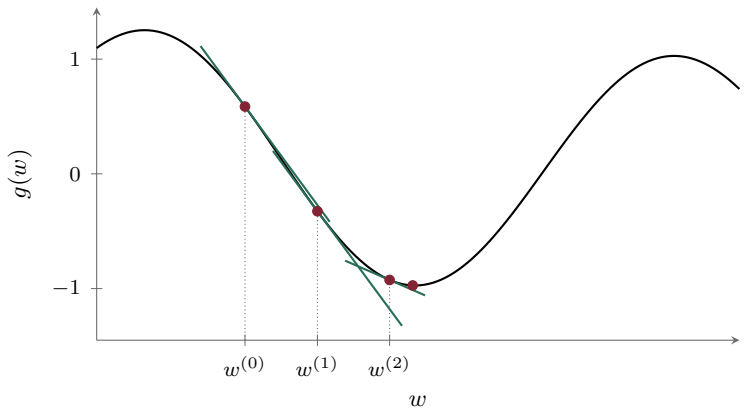


Figura 4.6. Tre passi di gradient descent su $g(w) = \sin(3w) + 0{,}1\,w^2$, da $w^{(0)} = -1{,}2$ con $\alpha = 0{,}1$. In ogni punto la retta tangente, in verde, indica da che parte si scende; il passo è lungo α per la pendenza, e si accorcia man mano che la pendenza cala: $-1{,}2 \to -0{,}91 \to -0{,}61 \to -0{,}52$, verso il minimo globale in $-0{,}51$.

Il gradient descent è il metodo di ottimizzazione più usato: grazie alla differenziazione automatica il gradiente si calcola in fretta anche con moltissime variabili,

molto più che cercando a caso direzioni di discesa come i **metodi di ordine zero**. E $-\nabla g$ è sempre una direzione di discesa, quindi con una step size scelta bene ogni passo fa progressi.

Quando il gradiente diventa quasi nullo, $\nabla g(\mathbf{w}^{(k)}) \approx \mathbf{0}$, diventano quasi nulli anche i passi, e il punto smette di muoversi. Succede vicino ai punti stazionari: il gradient descent si ferma vicino a un minimo, globale o locale, oppure a una sella.

Se la funzione ha più valli, dove si arriva dipende dal punto di partenza $\mathbf{w}^{(0)}$ e dalla step size. Su $\sin(3w) + 0{,}1\,w^2$, con $\alpha = 0{,}05$ (Figura 4.7):

- da $w^{(0)} = 4{,}5$ la discesa si ferma nella valle più vicina, il minimo locale in $w \approx 3{,}59$, dove $g \approx 0{,}31$;
- da $w^{(0)} = -1{,}5$ arriva al minimo globale, in $w \approx -0{,}51$, dove $g \approx -0{,}97$.

Come l'escursionista nella nebbia, il gradiente conosce solo la pendenza nel punto in cui ci si trova, e non può sapere che altrove c'è una valle più profonda.

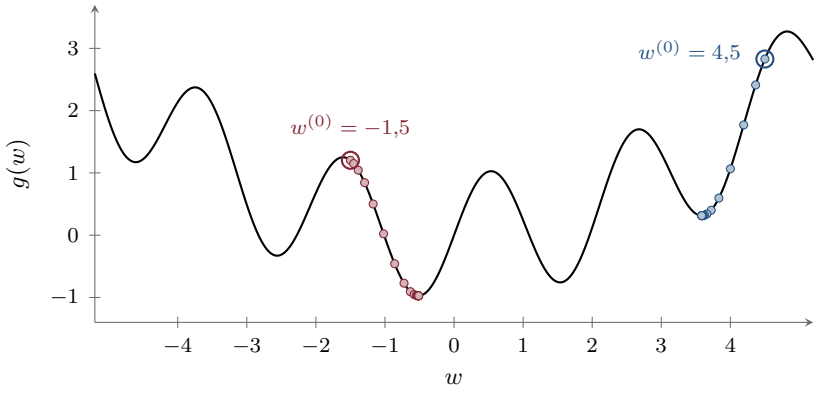


Figura 4.7. Venti passi di gradient descent su $g(w) = \sin(3w) + 0{,}1\,w^2$ con $\alpha = 0{,}05$, da 4,5 (in blu) e da $-1{,}5$ (in rosso); i cerchi grandi sono i punti di partenza.

La ciotola dell'esempio non ha questo problema. Una funzione è **convessa** se ha la forma di una ciotola, senza conche secondarie: il segmento che unisce due punti qualsiasi del grafico non passa mai sotto il grafico. In una funzione convessa ogni minimo locale è anche globale, e il gradient descent, con una step size adatta, arriva al minimo globale da qualunque punto parta.

4.6 Il passo e quando fermarsi

La step size decide quanto sono lunghi i passi dell'escursionista, e conta moltissimo. Se è troppo piccola la discesa è lentissima; se è troppo grande i passi scavalcano il fondo della conca e finiscono più in alto sul versante opposto, e la discesa può diventare una salita. Lo si vede bene su $g(w) = w^2$, che ha derivata $2w$, dove un passo è

$$w^{(k)} = w^{(k-1)} - \alpha \cdot 2w^{(k-1)} = (1 - 2\alpha)\, w^{(k-1)} :$$

a ogni passo w viene moltiplicato per lo stesso fattore $1-2\alpha$. Partendo da $w^{(0)} = -2{,}5$ (Figura 4.8):

- con $\alpha = 0{,}05$ il fattore è 0,9: dopo cinque passi $w \approx -1{,}48$ e $g \approx 2{,}18$, ancora lontano dal minimo;
- con $\alpha = 0{,}2$ il fattore è 0,6: dopo cinque passi $w \approx -0{,}19$ e $g \approx 0{,}04$;
- con $\alpha = 1{,}2$ il fattore è $-1{,}4$: w salta da una parte all'altra del minimo, ogni volta più lontano, e g cresce: 6,25, 12,25, 24, 47, 92, 181.

Qui la regola è semplice: con $\alpha = 0{,}5$ il fattore è zero e si arriva al minimo in un passo solo, con $\alpha > 1$ il fattore è minore di -1 e la discesa diverge.

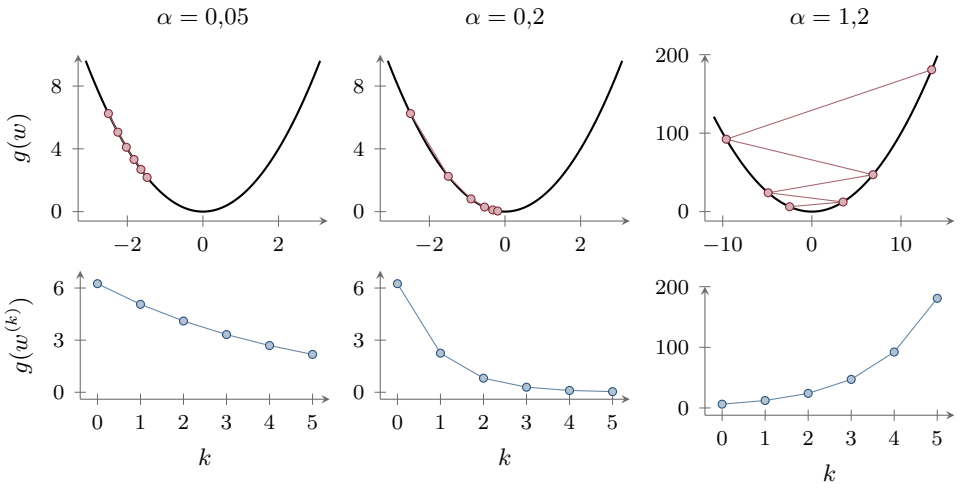


Figura 4.8. Gradient descent su $g(w) = w^2$ da $w^{(0)} = -2{,}5$ con tre step size: in alto i punti sulla parabola, uniti nell'ordine; in basso la storia del costo $g(w^{(k)})$ nei primi cinque passi (la scala del terzo pannello è diversa).

Per scegliere la step size ci sono due strategie di base:

- una **step size fissa**, $\alpha = 10^{-m}$ con m intero (1; 0,1; 0,01; ...), provando più valori e tenendo il migliore;
- una **step size decrescente**, $\alpha = 1/k$ al passo k: i passi si accorciano man mano.

La differenza si vede con il valore assoluto, $g(w) = |w|$, partendo da $w^{(0)} = 1{,}75$ (Tabella 4.2). La sua derivata è $+1$ per $w > 0$ e -1 per $w < 0$: la pendenza è la stessa ovunque, quindi ogni passo è lungo esattamente α, anche vicino al minimo. Con $\alpha = 0{,}5$ fissa la discesa resta incastrata: dopo tre passi oscilla tra $-0{,}25$ e 0,25 e non si avvicina mai di più. Con $\alpha = 1/k$ i passi si accorciano, e i punti, pur saltando da una parte all'altra, si avvicinano sempre più a 0.

k	0	1	2	3	4	5	6
$w^{(k)}$ con $\alpha = 0{,}5$	1,75	1,25	0,75	0,25	$-0{,}25$	0,25	$-0{,}25$
$w^{(k)}$ con $\alpha = 1/k$	1,75	0,75	0,25	$-0{,}083$	0,167	$-0{,}033$	0,133

Tabella 4.2. Gradient descent su $g(w) = |w|$ da $w^{(0)} = 1{,}75$, con step size fissa e decrescente.

Il grafico del costo $g(\mathbf{w}^{(k)})$ in funzione del passo k è la **storia del costo** (*cost function history plot*), come quelli in basso nella Figura 4.8. Serve a due cose:

- a scoprire i problemi: un costo che cresce segnala una divergenza, uno che sale e scende un'oscillazione, uno che resta piatto troppo presto una discesa bloccata;
- a scegliere la step size e gli altri **iperparametri**, per avere una discesa stabile e veloce. I parametri di un modello, come a e b nella retta del Capitolo 1, li trova l'ottimizzazione; gli iperparametri, come α, non si imparano: li fissa prima chi lancia l'algoritmo.

Con molte variabili la funzione non si può disegnare, e la storia del costo è spesso l'unico modo pratico di seguire l'ottimizzazione.

La storia del costo non deve per forza scendere a ogni passo: conta arrivare al valore più basso possibile, e un po' di oscillazione è accettabile, a volte utile, se aiuta a uscire da una valle poco profonda. Per esempio

$$g(\mathbf{w}) = w_1^2 + w_2^2 + 2\sin^2\big(1{,}5\,(w_1 + w_2)\big) + 2,$$

dove $\sin^2 x$ è il quadrato di $\sin x$, è una ciotola con delle increspature: ha il minimo globale in $(0, 0)$, dove vale 2, e dei minimi locali. Partendo da (3; 2,5), dove $g \approx 18{,}95$:

- con $\alpha = 0{,}01$ la discesa è regolare ma lentissima: dopo dieci passi $g \approx 11{,}6$;
- con $\alpha = 0{,}1$ è regolare e si ferma in un minimo locale, con $g \approx 3{,}97$;
- con $\alpha = 0{,}2$ il costo sale e scende (10,68; 6,44; 2,63; 4,20; 2,11; 2,81; 4,30), ma all'ottavo passo arriva a 2,00, il minimo globale.

Resta da decidere quando fermarsi: vicino a un punto stazionario i passi diventano quasi nulli, ma non si annullano mai del tutto. In pratica si usa uno di questi **criteri di arresto**, dove ε è una soglia piccola scelta da chi usa l'algoritmo e N è il numero di variabili:

1. *sul gradiente*: ci si ferma quando $\|\nabla g(\mathbf{w}^{(k-1)})\|_2$ è abbastanza piccolo, perché si è vicini a un punto stazionario;
2. *sul passo*: ci si ferma quando $\frac{1}{N}\,\|\mathbf{w}^{(k)} - \mathbf{w}^{(k-1)}\|_2 \le \varepsilon$, perché i passi non portano più da nessuna parte;
3. *sul costo*: ci si ferma quando $\frac{1}{N}\,\big|g(\mathbf{w}^{(k)}) - g(\mathbf{w}^{(k-1)})\big| \le \varepsilon$, perché il costo cambia ormai di pochissimo;
4. *pratico*: ci si ferma dopo un numero massimo di passi, fissato in anticipo.

Il fattore $1/N$ tiene conto del numero di variabili, così la stessa soglia va bene per problemi di dimensioni diverse.

4.7 I limiti del gradient descent

Ogni gradiente ha una direzione e una lunghezza, e da ciascuna viene un problema: la direzione può cambiare a ogni passo, e la discesa procede a zig-zag (*zig-zagging*); la lunghezza può diventare piccolissima, e la discesa avanza a passo di lumaca (*slow-crawling*).

4.7.1 Lo zig-zag

Per lo zig-zag servono le curve di livello della Sezione 4.3: come l'acqua che scorre giù da un pendio, la discesa più ripida le taglia ad angolo retto.

Nella ciotola le curve di livello sono cerchi, la perpendicolare punta dritta al centro, e i passi vanno dritti al minimo (Figura 4.9, a sinistra). In una *valle stretta*, invece, le curve di livello sono ellissi molto allungate, quasi parallele tra loro, e la perpendicolare punta verso la parete opposta della valle più che lungo la valle: i passi rimbalzano da una parete all'altra, e lungo la valle si avanza lentamente. L'escursionista, in un canalone stretto, sente soprattutto la pendenza delle pareti, e non quella, molto più debole, lungo il fondo del canalone.

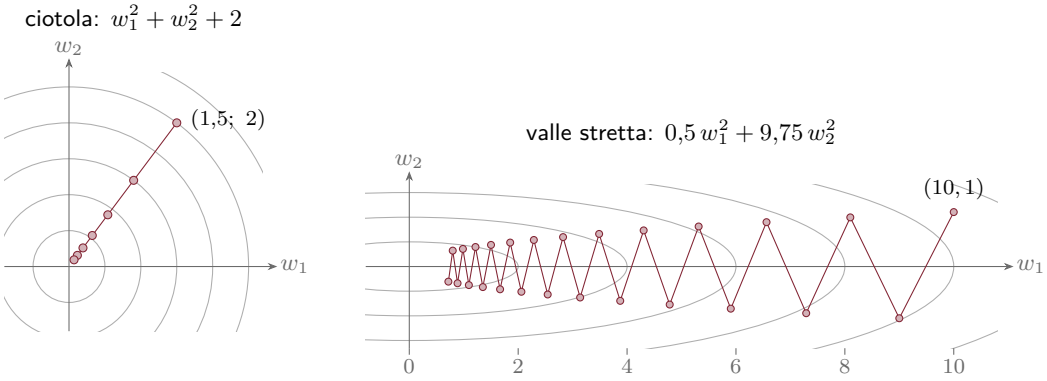


Figura 4.9. Curve di livello e passi del gradient descent, con la stessa scala sui due assi di ogni pannello: a sinistra la ciotola dell'esempio della Sezione 4.5, da (1,5; 2) con $\alpha = 0{,}2$; a destra la valle $0{,}5\,w_1^2 + 9{,}75\,w_2^2$, da $(10, 1)$ con $\alpha = 0{,}1$, per 25 passi.

Un esempio è la funzione $g(\mathbf{w}) = \mathbf{w}^\top \mathbf{A}\mathbf{w}$, una forma quadratica; con una matrice $\mathbf{A}$ diagonale è la somma dei quadrati delle componenti, ciascuno moltiplicato per il suo elemento della diagonale:

$$\mathbf{A} = \begin{bmatrix} 0{,}5 & 0 \\ 0 & 9{,}75 \end{bmatrix}, \qquad g(\mathbf{w}) = 0{,}5\,w_1^2 + 9{,}75\,w_2^2.$$

È una valle con il fondo in $(0, 0)$, lunga nella direzione di w_1 e stretta in quella di w_2: allontanandosi dal fondo, g cresce quasi venti volte più in fretta lungo w_2 che lungo w_1. Le derivate parziali sono w_1 e $19{,}5\,w_2$, e con $\alpha = 0{,}1$ un passo moltiplica ogni componente per il suo fattore:

$$w_1^{(k)} = w_1^{(k-1)} - 0{,}1 \cdot w_1^{(k-1)} = 0{,}9\,w_1^{(k-1)},$$
$$w_2^{(k)} = w_2^{(k-1)} - 0{,}1 \cdot 19{,}5\,w_2^{(k-1)} = -0{,}95\,w_2^{(k-1)}.$$

Partendo da $(10, 1)$:

- w_2 cambia segno a ogni passo e si riduce appena, 1, $-0{,}95$, 0,90, $-0{,}86$, …: è lo zig-zag da una parete all'altra;
- w_1 si riduce del 10% a ogni passo, e dopo 25 passi vale circa 0,72.

Più la valle è allungata, peggio va. Si ripeta la stessa discesa, con lo stesso α e dallo stesso punto, cambiando solo il numero 0,5 in **A**: più è piccolo, più il fondo della valle è piatto lungo w_1, e più lenta è l'avanzata (Tabella 4.3). Lo zig-zag di w_2 resta quello di prima, perché 9,75 non è cambiato.

Coefficiente di w_1^2	Fattore di w_1 a ogni passo	w_1 dopo 25 passi
0,5	0,9	0,72
0,1	0,98	6,0
0,01	0,998	9,5

Tabella 4.3. La stessa discesa, da $(10, 1)$ con $\alpha = 0{,}1$, in tre valli sempre più allungate. Con coefficiente c la derivata parziale rispetto a w_1 è $2c\,w_1$, quindi un passo moltiplica w_1 per $1 - 0{,}2\,c$.

Ridurre α attenua lo zig-zag, ma rallenta ancora di più l'avanzata lungo la valle: il problema non si risolve.

4.7.2 Lo slow-crawling

Dove la funzione è molto piatta, i passi si accorciano ben prima di arrivare al minimo: è l'escursionista su un tratto quasi pianeggiante, dove il terreno non dice più da che parte andare. Per esempio $g(w) = w^4 + 0{,}1$ ha il minimo in 0 e derivata $4w^3$, che vicino a 0 è piccolissima perché la funzione è molto piatta. Partendo da $w^{(0)} = -1$ con $\alpha = 0{,}1$, il primo passo è lungo 0,4 e porta a $-0{,}6$; poi i passi si accorciano, 0,086, 0,054, 0,039, …, e dopo dieci passi si è appena a $-0{,}30$, con passi di poco più di 0,01.

Con le funzioni non convesse c'è un rischio in più: la discesa può fermarsi su una sella e non raggiungere mai il minimo globale. Si pensi a una funzione che, partendo da $w = 0$, scende fino a un tratto orizzontale in $w \approx 0{,}23$, dove vale 1, e solo dopo riprende a scendere, fino al minimo in $w = 0{,}5$, dove vale circa 0,48. Partendo da $w = 0$, la discesa avanza sempre più piano verso il tratto orizzontale, dove la derivata si annulla, e lì si ferma: il minimo, poco più avanti, non viene raggiunto.

4.8 La PCA con il gradient descent

Nel Capitolo 3 la base della PCA si trovava con una formula, gli autovettori della matrice di covarianza. La si può trovare anche con il gradient descent, in due modi, uno per ciascuna delle due letture di quel capitolo: massimizzare la varianza oppure minimizzare l'errore di ricostruzione. In entrambi c'è un ingrediente nuovo, un **vincolo**: le colonne di **C**, la matrice $N \times K$ che ha in colonna i K vettori della base, devono restare ortonormali, $\mathbf{C}^\top \mathbf{C} = \mathbf{I}_K$, dove $\mathbf{I}_K$ è la matrice identità $K \times K$. Un passo di discesa allunga e inclina le colonne, e le porterebbe fuori dal vincolo; per questo dopo ogni passo le colonne si **ortonormalizzano**, cioè si rendono di nuovo di lunghezza 1 e perpendicolari tra loro. Con $K = 1$ basta dividere la colonna per la sua lunghezza. Con più colonne si usa il procedimento di **Gram–Schmidt**: si normalizza la prima colonna, poi a ogni colonna successiva si toglie la sua «ombra» sulle precedenti, così che diventi perpendicolare a tutte, e la si normalizza. Con le

colonne $(3,4)^\top$ e $(1,0)^\top$, per esempio, la prima diventa $(0{,}6;\ 0{,}8)^\top$; l'ombra della seconda su di essa è 0,6, e togliendola resta $(1,0)^\top - 0{,}6\,(0{,}6;\ 0{,}8)^\top = (0{,}64;\ -0{,}48)^\top$, lungo 0,8, che normalizzato dà $(0{,}8;\ -0{,}6)^\top$.

4.8.1 Massimizzare la varianza

Nel Capitolo 3 la varianza dei dati lungo una direzione $\mathbf{c}$ di lunghezza 1 era $\frac{1}{P}\,\mathbf{c}^\top\mathbf{X}\mathbf{X}^\top\mathbf{c}$, con i P punti centrati nelle colonne di $\mathbf{X}$. Con K direzioni, le colonne $\mathbf{c}_1,\dots,\mathbf{c}_K$ di $\mathbf{C}$, la varianza catturata in tutto è la somma delle varianze lungo ciascuna. La si scrive con la **traccia**, tr, la somma degli elementi sulla diagonale di una matrice quadrata: la matrice $\mathbf{C}^\top\mathbf{X}\mathbf{X}^\top\mathbf{C}$, che è $K\times K$, ha sulla diagonale proprio i numeri $\mathbf{c}_n^\top\mathbf{X}\mathbf{X}^\top\mathbf{c}_n$, quindi

$$\operatorname{tr}(\mathbf{C}^\top\mathbf{X}\mathbf{X}^\top\mathbf{C}) = \sum_{n=1}^{K}\mathbf{c}_n^\top\mathbf{X}\mathbf{X}^\top\mathbf{c}_n$$

è P volte la varianza catturata dalle K direzioni. Il problema diventa

$$\max_{\mathbf{C}}\ \operatorname{tr}(\mathbf{C}^\top\mathbf{X}\mathbf{X}^\top\mathbf{C}) \quad\text{con}\quad \mathbf{C}^\top\mathbf{C} = \mathbf{I}_K.$$

Massimizzare la traccia equivale a minimizzarne l'opposto (Sezione 4.1), e un passo di discesa sull'opposto è

$$\mathbf{C}^{(k)} = \mathbf{C}^{(k-1)} + \alpha\,\mathbf{X}\mathbf{X}^\top\mathbf{C}^{(k-1)},$$

seguito dall'ortonormalizzazione delle colonne. Il gradiente ha anche un fattore 2, che si assorbe nella step size. Il passo aggiunge a ogni colonna la sua trasformata tramite $\mathbf{X}\mathbf{X}^\top = P\,\Sigma$, dove Σ è la matrice di covarianza. Questa trasformazione allunga di più le direzioni in cui i dati variano di più, gli autovettori con gli autovalori più grandi: passo dopo passo la colonna ruota verso la prima componente principale.

Esempio (la PCA per passi). Si riprendano i tre punti centrati della Sezione 3.8, $(1,1)$, $(-1,0)$ e $(0,-1)$, per cui $\mathbf{X}\mathbf{X}^\top = \left[\begin{smallmatrix}2 & 1\\ 1 & 2\end{smallmatrix}\right]$, la somma dei tre prodotti esterni calcolata lì. Con $K=1$, $\alpha=1$ e la colonna di partenza $\mathbf{c}^{(0)} = (1,0)^\top$:

1. $\mathbf{c}^{(0)} + \mathbf{X}\mathbf{X}^\top\mathbf{c}^{(0)} = (1,0)^\top + (2,1)^\top = (3,1)^\top$, che diviso per la sua lunghezza, $\sqrt{10}$, dà $\mathbf{c}^{(1)} \approx (0{,}95;\ 0{,}32)^\top$, inclinato di 18° sull'asse orizzontale;
2. il passo successivo dà $\mathbf{c}^{(2)} \approx (0{,}86;\ 0{,}51)^\top$, a 31°;
3. poi vengono $(0{,}79;\ 0{,}61)^\top$ a 38° e $(0{,}75;\ 0{,}66)^\top$ a 41°.

L'angolo si avvicina a 45°: la colonna tende a $\mathbf{v}_1 = (1,1)^\top/\sqrt{2}$, la prima componente principale trovata nella Sezione 3.8. Lì la traccia vale $\mathbf{v}_1^\top\mathbf{X}\mathbf{X}^\top\mathbf{v}_1 = 3$, cioè $P\lambda_1 = 3\cdot 1$, dove $\lambda_1 = 1$ è l'autovalore di $\mathbf{v}_1$, la varianza lungo quella direzione.

4.8.2 Minimizzare l'errore di ricostruzione

L'errore di ricostruzione si scrive in forma compatta con la **norma di Frobenius** di una matrice $\mathbf{B}$, $\|\mathbf{B}\|_F$: la radice della somma dei quadrati di tutti i suoi elementi,

cioè la lunghezza della matrice vista come un unico lungo vettore. Per esempio la norma di $\left[\begin{smallmatrix}1 & 2\\ 2 & 0\end{smallmatrix}\right]$ è $\sqrt{1+4+4+0} = 3$. Le colonne di $\mathbf{C}\mathbf{C}^\top\mathbf{X} - \mathbf{X}$ sono gli errori dei singoli punti, $\mathbf{C}\mathbf{C}^\top\mathbf{x}_p - \mathbf{x}_p$, quindi

$$L(\mathbf{C}) = \|\mathbf{C}\mathbf{C}^\top\mathbf{X} - \mathbf{X}\|_F^2 = \sum_{p=1}^{P} \|\mathbf{C}\mathbf{C}^\top\mathbf{x}_p - \mathbf{x}_p\|_2^2$$

è P volte l'errore medio $g(\mathbf{C})$ della Sezione 3.2. Quando $\mathbf{C}$ ha colonne ortonormali, come dopo ogni ortonormalizzazione, il gradiente rispetto a $\mathbf{C}$, cioè la matrice delle derivate parziali di L rispetto ai singoli elementi di $\mathbf{C}$, è

$$\nabla_{\mathbf{C}} L(\mathbf{C}) = 2\,(\mathbf{C}\mathbf{C}^\top\mathbf{X} - \mathbf{X})\mathbf{X}^\top\mathbf{C},$$

e il passo è quello solito, seguito dall'ortonormalizzazione:

$$\mathbf{C}^{(k)} = \mathbf{C}^{(k-1)} - \alpha\,\nabla_{\mathbf{C}} L\big(\mathbf{C}^{(k-1)}\big).$$

Le due strade portano allo stesso risultato, perché minimizzare l'errore di ricostruzione equivale a massimizzare la varianza (Sezione 3.4), e arrivano allo stesso sottospazio della formula del Capitolo 3 (Tabella 4.4).

	Autovettori	Varianza con il gradient descent	Errore con il gradient descent
che cosa si cerca	i primi K autovettori di Σ	il massimo di $\mathrm{tr}(\mathbf{C}^\top\mathbf{X}\mathbf{X}^\top\mathbf{C})$	il minimo di $\|\mathbf{C}\mathbf{C}^\top\mathbf{X} - \mathbf{X}\|_F^2$
come si arriva a $\mathbf{C}$	con una formula, in una volta sola	a passi	a passi
che cosa fa un passo	–	aggiunge a $\mathbf{C}$ il termine $\alpha\,\mathbf{X}\mathbf{X}^\top\mathbf{C}$	toglie da $\mathbf{C}$ il termine $\alpha\,\nabla_{\mathbf{C}} L(\mathbf{C})$
colonne ortonormali	lo sono già	si ortonormalizzano dopo ogni passo	si ortonormalizzano dopo ogni passo

Tabella 4.4. Tre modi di trovare la base della PCA.

Nell'esempio dei tre punti, con $\mathbf{C} = \mathbf{v}_1$, l'errore vale $L = 1$, cioè $P\lambda_2 = 3 \cdot \frac{1}{3}$, dove $\lambda_2 = \frac{1}{3}$ è il secondo autovalore: P volte la varianza scartata.